

Highlights

CPAI: A Controlled-Point Discriminant Analysis for IoT Network Intrusion Detection

Van-Hoi Nguyen, Xuan-Ha Nguyen, Kim-Hung Le

- A closed-form controlled-point discriminant framework (CPAI) that resolves the singularity problem of null-space discriminant methods such as NFST and KNFST.
- Two training strategies: OvA for simplicity and OvR for accuracy.
- Evaluation on seven IoT intrusion detection benchmarks, including Edge-IIoTset (15 classes) and 5G-NIDD, with CPAI-OvR reaching a cross-dataset mean MCC of 0.9722.
- Sub-second inference latency and peak memory below 3 GB on every benchmark, which fits Raspberry-Pi-class IoT hardware without a GPU.

CPAI: A Controlled-Point Discriminant Analysis for IoT Network Intrusion Detection

Van-Hoi Nguyen^{a,b,1}, Xuan-Ha Nguyen^{a,b,1} and Kim-Hung Le^{a,b,*}

^aUniversity of Information Technology, Ho Chi Minh City, Vietnam

^bVietnam National University, Ho Chi Minh City, Vietnam

ARTICLE INFO

Keywords:

Anomaly-based NIDS
Controlled-point discriminant analysis
Feature extraction; Fisher criterion
Foley-Sammon transform
IoT intrusion detection
Kernel methods
Linear discriminant analysis
Null Foley-Sammon transform
Regularised least squares in RKHS
Singularities.

ABSTRACT

The proliferation of Internet of Things (IoT) devices poses significant security challenges, particularly for network intrusion detection, due to evolving cyber attacks and resource constraints of IoT devices. While existing distance-based approaches like Null Foley-Sammon Transform (NFST) show promise, they suffer from singularity and small sample size problems, limiting their effectiveness in diverse IoT scenarios. In this paper, we introduce Controlled-Point Discriminant Analysis (CPAI), a novel algorithm designed for robust IoT network intrusion detection. CPAI addresses both problems by controlling where each class is placed in the projection space and by reformulating the learning problem as a regularised kernel system with a closed-form solution. The kernel formulation also handles non-linear class structure and removes the small-sample-size constraint. To cover different deployment needs, we provide two training strategies: One-vs-All (OvA) for computational efficiency and One-vs-Rest (OvR) for enhanced accuracy. Extensive evaluation across seven benchmark IoT network security datasets shows that CPAI outperforms KNFST and other distance-based algorithms in accuracy and stability. The CPAI-OvR variant achieves over 98% accuracy on most datasets, with a minimum of 98.15% on the most challenging scenarios, while consistently maintaining false positive rates below 1.158%.

1. Introduction

The Internet of Things (IoT) has revolutionized various critical sectors (e.g., healthcare, logistics, and manufacturing) by enabling seamless communication and automation over the Internet. While this significantly improves the efficiency of operations and services, it also introduces cybersecurity threats by exposing networks to unauthorized access, data breaches, and service disruptions [2]. A closely related concern is data privacy: IoT network flows carry behavioral signals about end users that raise privacy risks comparable to those documented in the social-network data-sharing literature [15, 12]. These combined security and privacy concerns motivate a preference for edge-based detection, which keeps sensitive traffic on the local gateway instead of sending it to the cloud. However, IoT devices often have limited computational and energy resources, making them unsuitable for computationally intensive security mechanisms. Therefore, developing lightweight yet effective security solutions specifically designed for IoT systems is a critical necessity [25]. This paper aims to advance this direction by proposing a security solution that is accurate across diverse IoT attack patterns, deployable on commodity IoT-gateway hardware without a GPU, and grounded in a theoretically principled classification framework.

*Corresponding author

✉ hoinv@uit.edu.vn (V. Nguyen); hanx@uit.edu.vn (X. Nguyen);
hunglk@uit.edu.vn (K. Le)

ORCID(s):

¹These authors contributed equally to this work.

1.1. Paradigms in Anomaly-Based Intrusion Detection

Anomaly-based Network Intrusion Detection Systems (aNIDS) represent a promising solution for this challenge. Their primary strength lies in their adaptability to automatically identify threats by analyzing deviations from normal network behavior, making them more flexible than signature-based systems that rely on predefined attack patterns. Numerous studies have demonstrated the superior performance of aNIDS in conventional networks [2, 26, 36]. Within the domain of aNIDS, several methodological paradigms have been dominant.

Among these paradigms, deep learning (DL) and tree-based algorithms are widely used in proposed aNIDSs due to their effectiveness in processing tabular network traffic data [2, 1, 6]. DL models excel at automatically capturing complex patterns [38]. However, they typically demand substantial computational resources (e.g., GPUs, significant memory), making them impractical for resource-constrained IoT devices. Even though inference can be optimized for edge devices (e.g., via model distillation), the initial training overhead remains a significant barrier. While tree-based models are generally more lightweight, they can struggle in unsupervised or semi-supervised settings common in IoT, where labeled data for novel attacks is limited.

Given these trade-offs, distance-based algorithms have emerged as a promising alternative for IoT network security. These algorithms do not require specialized hardware like GPUs and are highly adaptable in contexts where attack labels are unavailable, making them more suitable for resource-constrained IoT devices [16]. This has led to a renewed interest in methods rooted in Linear Discriminant Analysis (LDA) [19], which offer a powerful middle ground:

they provide a principled approach to feature extraction that is both computationally efficient and more interpretable than deep learning "black boxes". Its key concept involves projecting instances from a high-dimensional space into a lower-dimensional subspace through linear transformation, thereby compacting samples and making them easier to classify. Building on this foundation, the next section reviews the most prominent discriminant-analysis method in this line of work and identifies the two theoretical flaws that keep it from working reliably in IoT settings.

1.2. The Challenge of Null-Space Discriminant Analysis

A state-of-the-art method in this direction is the Null Foley-Sammon Transform (NFST) [23], that extends classical Linear Discriminant Analysis [19, 20]. The theoretical elegance of NFST lies in its objective to project all samples of a class to a single, unique point by enforcing zero within-class scatter, thereby maximizing discrimination according to the Fisher criterion. Despite its strong theoretical appeal, our analysis and recent literature reveal two critical, unresolved flaws that severely hinder NFST's reliability in practical IoT scenarios:

- *Singularity problem:* The foundational assumption that projected class points remain distinct is not guaranteed. However, as demonstrated by Ali and Chaudhuri [3], these singularities can erroneously overlap, leading to catastrophic misclassification. The problem gets worse in IoT networks where normal and malicious behaviors could have several similar network characteristics, reducing the model's accuracy. To the best of our knowledge, this singularity problem remains unresolved within the application of aNIDS in IoT networks.
- *Small sample size problem:* The second theoretical need of NFST is the linearity of sample vectors. This assumption yields that the number of samples (n) does not exceed the feature dimensionality (d), i.e., $n \leq d$. Consequently, applying NFST in IoT networks may reduce accuracy or even make it impossible to find a solution.

1.3. Our Contribution: Controlled-Point Discriminant Analysis

Recognizing these critical limitations, we propose a Controlled-Point discriminant Analysis for IoT (CPAI), a novel framework designed to overcome the theoretical drawbacks of NFST while retaining the strengths of discriminant analysis. The core innovation of CPAI is its ability to *explicitly control the placement of projected data points*, a principle that fundamentally resolves the singularity problem. Furthermore, by leveraging the kernel trick, CPAI handles non-linear class structure and removes the $n \leq d$ constraint on the number of samples.

Crucially, formulating CPAI as a regularized least-squares problem yields a *closed-form analytical solution*.

This transforms the learning problem into solving a single system of linear equations. This structural simplicity not only makes the model easier to analyze but also avoids potential issues like local minima and convergence problems inherent in more complex optimizers.

Together, the closed-form solution and the kernel-based geometric control give CPAI the computational efficiency and high accuracy required by IoT security deployments, which demand *robustness, efficiency, and reliability*. To enhance its flexibility for diverse deployment needs, we propose two training strategies: CPAI-OvA for computational efficiency, and CPAI-OvR for superior accuracy. We conduct extensive experiments on seven benchmark IoT datasets, demonstrating that our proposed CPAI framework significantly outperforms not only its predecessor (KNFST) but also a wide range of other strong baseline methods.

The main contributions of this paper are:

- **A new controlled-point discriminant framework.** CPAI resolves the singularity problem of NFST by explicitly setting the projection coordinate of each class. Training reduces to solving a single regularised kernel system, which has a closed-form solution and does not need a GPU. Kernelization also removes the $n \leq d$ small-sample-size constraint of linear null-space methods.
- **Two training strategies with clear decision rules.** We provide two complementary variants: CPAI-OvA, which uses a single scalar discriminant for speed, and CPAI-OvR, which uses c binary discriminants for higher accuracy. For each variant, we give an explicit test-time decision rule and a simple anomaly-detection threshold.
- **Extensive evaluation.** We compare CPAI to eight classical discriminant and distance-based baselines and four modern tabular baselines on seven IoT benchmarks, and report diverse metrics for both accuracy comparison and practical deployment.

The remainder of this paper is organized as follows. Section 3 details the theoretical foundations and implementation of our proposed CPAI framework. A direct comparison between CPAI and its predecessor, NFST, is provided in Appendix B to highlight our key theoretical advancements. Section 5 presents the comprehensive experimental setup, results, and analysis, validating CPAI's superior performance against a wide range of competitors. Finally, Section 7 concludes the paper with a summary of our findings and discusses directions for future work. To ensure the paper is self-contained, an overview of the NFST algorithm is provided in Appendix A.

2. Related Work

Network Intrusion Detection Systems (NIDS) for IoT have been approached from several methodological directions. We survey the principal families of methods, discuss

their practical limitations in resource-constrained IoT environments, and identify the gap that motivates the present work.

2.1. IoT Intrusion Detection: Problem Overview

Supervised multi-class classification represents a crucial functional element within broader IoT intrusion detection systems, enabling the precise identification and categorization of known attack patterns such as DDoS, botnet, reconnaissance, and data exfiltration [26, 36]. This framing distinguishes it from pure one-class anomaly detection and is the standard adopted by the five benchmark datasets used in this work [29, 4, 34, 37, 53].

IoT environments impose three compounding constraints that significantly restrict the applicability of many IDS methods [25]. First, IoT devices are resource-constrained: limited memory and processing power preclude the deployment of GPU-dependent models at the network edge. Second, IoT traffic is highly heterogeneous, spanning diverse protocols, device types, and attack patterns that vary across deployments. Third, the class imbalance inherent in IoT datasets, where benign traffic vastly outnumbers attack instances, can severely bias classifier performance [2, 26]. A comprehensive taxonomy of IDS approaches for IoT, organized by data source, detection technique, and placement strategy, is provided by Jamalipour and Murali [26]. In the following subsections, we review the major methodological paradigms within this taxonomy and assess their suitability for IoT gateway deployment.

2.2. Deep Learning and Tree-Based Approaches

Deep learning (DL) methods have achieved state-of-the-art detection performance in IoT IDS by automatically learning discriminative feature representations from raw network traffic [1, 2]. Convolutional Neural Networks (CNNs) exploit spatial correlations within traffic feature vectors, while Recurrent Neural Networks (RNNs)-particularly Long Short-Term Memory (LSTM) variants - capture temporal dependencies in sequential traffic flows [26]. Deep Autoencoders (AEs) offer an unsupervised alternative, learning a compact representation of normal traffic and identifying anomalies through reconstruction error. However, AE-based models face the challenge of representation collapse [41], where the network maps the entire feature manifold to a trivial point during training, losing all discriminative power. Ahmad et al. [1] provide a comprehensive benchmark of DL architectures in IoT IDS, confirming consistently strong detection rates. In the context of DDoS detection specifically, Cil et al. [14] propose a feed-forward DNN that integrates feature extraction and classification within a unified architecture, achieving 99.99% binary detection accuracy and 94.57% multi-class classification accuracy on the CICDDoS2019 dataset. A representative example of DL deployment under IoT hardware constraints is RealGuard [38], a lightweight DNN-based NIDS that operates directly on IoT gateways, achieving 99.57% average accuracy while processing over 10,000 packets per second on a Raspberry Pi. Despite these advances, DL methods share a fundamental limitation: their

dependence on GPU acceleration and large labeled training sets creates an inherent mismatch with the resource-constrained nature of IoT edge devices [26, 38]. Furthermore, their opaque internal representations reduce interpretability, which is critical in security-sensitive deployments.

Tree-based ensemble methods offer a more resource-efficient alternative. Decision Trees (DTs) provide interpretable, rule-based classification that naturally handles the heterogeneous feature types common in IoT network traffic [26]. Random Forest and XGBoost extend this principle by aggregating predictions from multiple DTs to improve robustness against noise and overfitting. Azam et al. [6] conduct a comparative evaluation of ML-based IDS approaches and demonstrate that tree-based ensembles achieve competitive detection accuracy with substantially lower computational overhead than DL counterparts. Nevertheless, tree-based models are sensitive to feature engineering quality and tend to underperform in semi-supervised settings where labeled data for novel attack classes is limited.

2.3. Distance-Based and Discriminant Analysis Methods

Existing detection methods can be further distinguished by their geometric interpretation of normality. Proximity-based methods such as K-Nearest Neighbor (KNN) [39] and Local Outlier Factor (LOF) [10] identify anomalies by measuring local spatial deviations from known normal behavior, while boundary-based methods including One-Class SVM (OCSVM) [45] and Support Vector Data Description (SVDD) [51] model normality as a minimal enclosing hyperplane or hypersphere. Falcao et al. [16] provide a quantitative comparison of these approaches in the IoT IDS context, confirming their effectiveness for unsupervised anomaly detection. However, proximity-based methods scale poorly with the number of training samples (e.g. KNN requires $\mathcal{O}(n)$ comparisons per inference), while boundary-based methods are sensitive to kernel selection and tend to ignore the intrinsic class structure of the data.

A more principled approach lies in discriminant analysis, which explicitly models the internal variance structure of traffic classes. Fisher's Linear Discriminant Analysis (LDA) [19] provides a theoretically grounded framework for class separability maximization through linear projection, and the Foley-Sammon Transform (FST) [20] extends it by finding a set of optimal, mutually orthogonal discriminant vectors. The Null Foley-Sammon Transform (NFST) [23] represents the state of the art in this direction: by enforcing zero within-class scatter ($\phi^T S_w \phi = 0$), NFST aims to collapse all samples of a class to a single, distinct projection point, theoretically maximizing the Fisher criterion in the null space of S_w . Kernelized variants (KNFST) [9, 32] extend this principle to non-linear settings via the kernel trick, enabling application in the $n > d$ regime common in IoT datasets. However, Ali and Chaudhuri [3] demonstrates that NFST's singularity points are not guaranteed to remain distinct: under certain data configurations, different

classes collapse onto the same projection coordinate, leading to catastrophic misclassification. This singularity problem is further exacerbated in IoT networks where normal and malicious traffic often share overlapping statistical characteristics. In addition, linear NFST is restricted to the small sample size regime ($n \leq d$), limiting its applicability when the number of training samples exceeds the feature dimensionality. To the best of our knowledge, these two limitations remain unresolved in the prior IoT IDS literature [3, 23]. CPAI is developed to resolve both by replacing the passive singularity discovery of NFST with an explicit, target-driven projection system.

2.4. Federated and Lightweight Approaches

The centralized training paradigm shared by the methods described above raises additional challenges in IoT deployments: aggregating raw network traffic from distributed devices introduces communication overhead and creates data privacy risks, particularly in sensitive industrial and healthcare IoT contexts [25]. Federated Learning (FL) addresses these concerns by enabling collaborative model training across distributed clients without sharing raw data. Buyuktanir et al. [11] provide a comprehensive survey of FL-based IDS solutions across IoT, healthcare, and industrial network domains, demonstrating that FL achieves competitive detection accuracy while substantially improving data privacy. The survey highlights that the integration of Generative Adversarial Networks (GANs) and hybrid DL architectures within FL frameworks represents a promising direction for improving robustness against non-IID data distributions, a persistent challenge in heterogeneous IoT environments. At the intersection of FL and resource efficiency, Bocekci and Yildiz [8] propose a Mutex-based federated architecture for the full fine-tuning of the TinyLlama-1.1B language model on resource-constrained edge clients. While their application domain targets NLP adversarial attack detection rather than network IDS, the proposed sequential GPU-access protocol and FP16 memory optimization offer relevant engineering principles for deploying learning systems under strict hardware constraints - a challenge equally applicable to IoT-edge IDS deployment.

2.5. Summary and Research Gap

Table 1 provides a structured comparison of the surveyed IDS works. The analysis reveals a persistent and unaddressed gap. DL methods achieve high accuracy but require hardware resources incompatible with IoT edge deployment. Tree-based methods are lightweight but require extensive labeled data and struggle in semi-supervised settings. Distance and boundary-based methods are hardware-agnostic but ignore class structure and scale poorly. Discriminant analysis methods, particularly NFST and KNFST, offer the most promising balance of theoretical rigor and efficiency, but are fundamentally compromised by the singularity problem and the $n \leq d$ constraint [3, 23]. While individual techniques like kernelization or target-driven mapping exist in other contexts, no existing framework in the lineage of null-space

discriminant analysis has simultaneously resolved the singularity problem while maintaining a closed-form, hardware-agnostic solution uniquely suitable for IoT gateways.

CPAI directly addresses this gap. By reformulating the discriminant analysis objective as an explicit target assignment problem, solving $(K + \lambda I)\alpha = \beta$ to force each class to a predefined, distinct projection coordinate, CPAI structurally prevents singularity collapse. Kernelization extends this to arbitrary data dimensionalities, removing the $n \leq d$ constraint. The closed-form solution requires no iterative optimization and no GPU, making CPAI uniquely aligned with the efficiency requirements of resource-constrained IoT security.

3. Our Proposed Approach

In this section, we present our proposed algorithm CPAI, detailing its theoretical foundations and implementation. We first introduce the key concepts underlying the algorithm, followed by a detailed description of its implementation. We then explain how we integrate kernel methods to overcome the small sample size problem, enhancing the approach's suitability for real-world IoT network security applications.

3.1. Linear Case

Suppose the dataset consists of n samples $\{x^i\}_{i=1}^n$ belonging to c classes, denoted as $\{x_1^i\}_{i=1}^{n_1}, \dots, \{x_c^i\}_{i=1}^{n_c}$, where n_j is the number of samples in the j^{th} class, and $x_j^i \in \mathbb{R}^d$ represents the i^{th} sample vector of the j^{th} class. The total number of samples is $n = \sum_{j=1}^c n_j$. Throughout this paper, we use the standard inner product $(x, y) = x^T y$, $\forall x, y \in \mathbb{R}^d$. The Euclidean norm or the length of vector x is denoted by $\|x\| = (x, x)^{\frac{1}{2}}$. The projection of the vector u onto a non-zero vector v is a scalar $(u, v)/\|v\|$.

This section primarily addresses the small-sample-size (SSS) case in the classical sense, where the number of data instances is less than or equal to the feature dimension (i.e., $n \leq d$), and assumes linear independence among sample vectors. The latter assumption is not merely a mathematical convenience but a fundamental prerequisite for the existence of a valid null space in NFST [23] and its subsequent extensions [32, 9, 3]. Consequently, our proposed method is established upon the following theorem.

Theorem 1. Suppose $\{x_1, \dots, x_n\} \subset \mathbb{R}^d$ are linearly independent. For any target projection vector $\beta = (b_1, \dots, b_n)^T \in \mathbb{R}^n$, there exists a unique vector $\theta \in \text{span}\{x_1, \dots, x_n\} := \{\gamma_1 x_1 + \gamma_2 x_2 + \dots + \gamma_n x_n, \gamma_i \in \mathbb{R}, i = 1, \dots, n\}$ such that $(\theta, u_i) = b_i$ for all $i = 1, \dots, n$.

PROOF. Let $\beta = (b_1, \dots, b_n)^T \in \mathbb{R}^n$ be an arbitrary fixed target vector. The proof ends if the linear equation below admits a unique solution θ within the subspace $\text{span}\{x_1, \dots, x_n\}$:

$$Q^T \theta = \beta, \quad (1)$$

where $Q = (x_1, x_2, \dots, x_n) \in \mathbb{R}^{d \times n}$. Since we seek a solution $\theta \in \text{span}\{x_1, \dots, x_n\}$, it can be expressed as a

Table 1

Representative IoT intrusion detection works organized by methodological category, with evaluation datasets and key strengths and limitations.

Work	Category	Evaluation Dataset(s)	Key Strength	Key Limitation
Ahmad et al. [1]	Deep Learning	BoT-IoT, N-BaloT, others	Comprehensive DL benchmark for IoT IDS; strong detection accuracy	GPU-dependent; high memory footprint
Cil et al. [14]	Deep Learning	CICDDoS2019	Feed-forward DNN achieving 99.99% binary detection accuracy	Single-domain dataset; limited attack diversity
RealGuard [38]	Deep Learning	CICIDS2017	Lightweight DNN deployable on IoT gateways; real-time inference	Requires fully labeled training data; limited to known attacks
Azam et al. [6]	Tree-Based	CICIDS2017, NSL-KDD	Interpretable; computationally efficient ensemble approach	Degrades in semi-supervised and class-imbalanced settings
Falcao et al. [16]	Distance Boundary	/	Rigorous quantitative comparison of anomaly detectors for IDS	Ignores class structure; poor scalability with n
Guo et al. [23]; Bodesheim et al. [9]	Discriminant Analysis	Face recognition benchmarks	Zero within-class scatter; kernelised variant handles $n > d$	Singularity problem [3]; $n \leq d$ for linear case
Buyuktanir et al. [11]	Federated Learning	IoT, healthcare, industrial	Privacy-preserving; distributed training without raw data sharing	Communication overhead; non-IID data challenges
CPAI (this work)	Discriminant + Kernel	BoT-IoT, IoTID20, ToN-IoT, N-BaloT, CIC-IoT2023, Edge-IIoTset and 5G-NIDD	Resolves singularity; closed-form; no GPU; handles $n > d$	Kernel and λ selection required; $\mathcal{O}(n^2)$ training

unique linear combination of the basis vectors $\{x_1, \dots, x_n\}$ (as they are linearly independent)

$$\theta = \alpha_1 x_1 + \dots + \alpha_n x_n = Q\alpha, \quad (2)$$

where $\alpha = (\alpha_1, \dots, \alpha_n)^T \in \mathbb{R}^n$ is the unique vector of coefficients. Substituting (1) with its basis expansion (2), one obtains

$$Q^T Q \alpha = \beta. \quad (3)$$

The existence and uniqueness of the solution θ depend on the existence and uniqueness of the solution α to (3). It remains to prove that the $n \times n$ matrix $Q^T Q$ is of full rank (i.e., invertible). To this end, consider the homogeneous system $Q^T Q \alpha' = 0$. Multiplying by $(\alpha')^T$ on the left, we get $(\alpha')^T Q^T Q \alpha' = 0$, which implies $(Q\alpha', Q\alpha') = \|Q\alpha'\|^2 = 0$. This holds if and only if $Q\alpha' = 0$. Since the columns of Q are linearly independent by assumption, the only solution to $Q\alpha' = 0$ is the trivial solution $\alpha' = 0$. It follows from linear algebra theory that $Q^T Q$ is of full rank and thus invertible.

Consequently, the system (3) has a unique solution α . Substituting this unique α back into (2) yields the unique solution θ within the subspace $\text{span}\{x_1, \dots, x_n\}$.

This theorem guarantees that, provided the linear independence of the sample vectors $\{x^i\}$, there exists a unique discriminant vector θ (within the span of these samples) that maps each training sample x_j^i to an arbitrarily predefined target value b_j . This allows us to force samples of the j^{th} class to map to a single target value (e.g., b_j), while samples

from different classes (i.e., $k \neq j$) can be mapped to different target values ($b_k \neq b_j$). Notably, these target projection values can be chosen arbitrarily.

Lemma 2. Let H_j denote the affine hull generated by the data samples of the j^{th} class, i.e.,

$$H_j = \{x = a_1 x_j^1 + a_2 x_j^2 + \dots + a_{n_j} x_j^{n_j} : \sum_{i=1}^{n_j} a_i = 1\}.$$

Let θ be a vector constructed according to Theorem 1 such that $(\theta, x_j^i) = b_j$ (a constant value) for all samples $i = 1, \dots, n_j$ of the j^{th} class. Then, for any vector $x \in H_j$, it holds that $(\theta, x) = b_j$.

PROOF. The proof is straightforward. For any $x \in H_j$, one has the following linear combination: $x = a_1 x_j^1 + \dots + a_{n_j} x_j^{n_j}$ for some coefficients $a_1, \dots, a_{n_j}$ such that $\sum_{i=1}^{n_j} a_i = 1$. By the linearity of the inner product and the construction of θ , we have $(\theta, x_j^i) = b_j$ for all $i = 1, \dots, n_j$. Substituting this into the equation above yields:

$$(\theta, x) = \sum_{i=1}^{n_j} a_i b_j = b_j \sum_{i=1}^{n_j} a_i = b_j$$

provided that $(\theta, x_j^i) = b_j$ for any $i = 1, \dots, n_j$, and $\sum_{i=1}^{n_j} a_i = 1$.

The above lemma ensures that any point within the affine hull of the samples of the j^{th} class maps to the same predefined target value b_j when projected onto the discriminant direction θ .

According to Theorem (1), discriminant directions θ_i can be constructed by selecting appropriate target vectors β^i . A transformation matrix is then defined using one or more such vectors. Let $\Phi = (\theta_1, \dots, \theta_k)$ denote a matrix whose columns consist of k discriminant vectors. The transformed data is given by $y = \Phi^T x$. With appropriately chosen β^i vectors, this transformation maps samples of the same class to a single target point (or vector) in the k -dimensional projection space, while ensuring that target points for different classes are distinct. This framework leads to two primary implementation strategies for CPAI as a classifier:

1. **One-vs-All (OvA) approach:** This strategy classifies all c classes using a single discriminant vector θ . The target vector $\beta \in \mathbb{R}^n$ is constructed such that all samples belonging to the j^{th} class are mapped to the same target value b_j , while ensuring that target values for different classes remain distinct (i.e., $b_k \neq b_j$ for all $k \neq j$). For instance, β can be defined as:

$$\beta = (\underbrace{b_1, \dots, b_1}_{n_1}, \dots, \underbrace{b_j, \dots, b_j}_{n_j}, \dots, \underbrace{b_c, \dots, b_c}_{n_c})^T, \quad (4)$$

where $b_1, \dots, b_c$ are predefined, distinct scalar values that govern the final projection coordinates. This concept is illustrated in Fig. 1, where samples from multiple classes are projected onto a single discriminant direction θ , effectively mapping them to well-separated points $b_1, b_2, \dots, b_c$ within the 1D discriminant space.

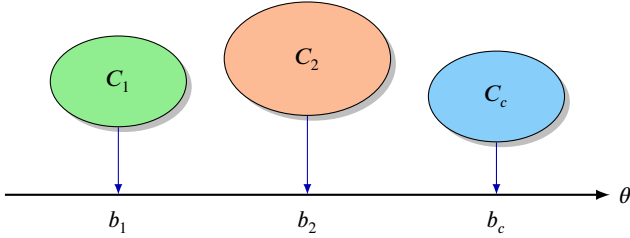


Figure 1: Visualization of the One-vs-All (OvA) strategy. Multiple class manifolds ($C_1, \dots, C_c$) are projected onto a single discriminant direction θ . By enforcing specific target coordinates b_j , the algorithm ensures that each class collapses into a distinct, non-overlapping point in the projected space.

2. **One-vs-Rest (OvR) approach:** This strategy decomposes the multi-class problem into multiple binary classification problems. A common variant is **One-vs-Rest (OvR)** where c separate discriminant vectors $\{\theta_1, \dots, \theta_c\}$ are constructed. For each j^{th} class, a discriminant vector θ_j is found such that samples of this class project to one value (e.g., 1), while samples from all other classes project to a different value (e.g., 0). This is achieved by setting c different target vectors $\beta^1, \dots, \beta^c$. For the j^{th} direction θ_j , the corresponding target vector $\beta^j \in \mathbb{R}^n$ is predefined as:

$$(\beta^j)_i = \begin{cases} 1 & \text{if sample } i \in \text{class } j, \\ 0 & \text{if sample } i \notin \text{class } j. \end{cases}$$

Equivalently, the target vector is expressed as:

$$\beta^j = (\underbrace{0, \dots, 0}_{n_1}, \dots, \underbrace{1, \dots, 1}_{n_j}, \dots, \underbrace{0, \dots, 0}_{n_c})^T. \quad (5)$$

By explicitly enforcing these binary targets, we provide a structural safeguard against class merging (singularity), ensuring each class remains linearly separable in the discriminant space. This concept is visualized in Fig. 2, where multiple discriminant vectors are utilized to partition the feature space into distinct class-specific regions.

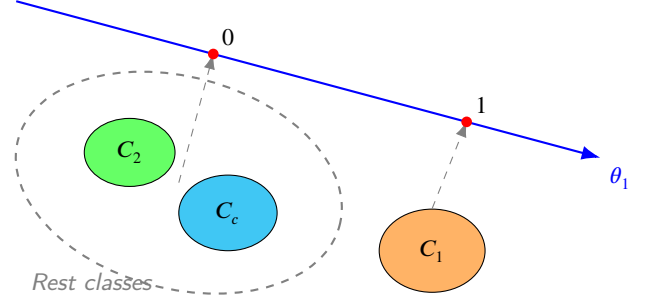


Figure 2: Illustration of the One-vs-Rest (OvR) strategy for class C_1 . The target class C_1 (orange) is projected onto the discriminant vector θ_1 to a specific target point 1, while the remaining classes ($C_2, \dots, C_c$, grouped by the dashed ellipse) are mapped to a different target 0. This explicit control of projection points prevents the class-merging singularity problem.

3.2. Nonlinear Case - Application of Kernel Trick

The linear CPAI formulation presented in Section 3.1 relies on the assumption of linear independence of the sample vectors $\{x^i\}_{i=1}^n \subset \mathbb{R}^d$. This assumption directly implies $n \leq d$: by fundamental linear algebra, the maximum number of linearly independent vectors in $\mathbb{R}^d$ is exactly d . Formally, $\text{rank}(Q) \leq \min(n, d)$ for the data matrix $Q = (x^1, \dots, x^n) \in \mathbb{R}^{d \times n}$, so the Gram matrix $Q^T Q$ is invertible only when $n \leq d$. When $n > d$, $Q^T Q$ is rank-deficient and the linear system $Q^T Q \alpha = \beta$ has no unique solution. This scenario corresponds to the small-sample-size case often encountered in high-dimensional feature spaces. Such assumptions may not hold in many real-world problems, particularly when $n > d$ or the underlying class structures are non-linear, limiting the applicability of the linear theory. To overcome these limitations and to allow for more flexibility in the proposed model, the kernel trick can be employed, allowing the analysis to be performed implicitly in a potentially higher-dimensional Reproducing Kernel Hilbert Space (RKHS) $\mathcal{H}$. We map features via a nonlinear function $\Psi : \mathcal{X} \rightarrow \mathcal{H}$, where the inner product is defined by a kernel function: $(\Psi(x^i), \Psi(x^l))_{\mathcal{H}} = \kappa(x^i, x^l)$.

To provide a rigorous justification for our kernelized formulation, we invoke the Representer Theorem [44]. This theorem states that for any regularized empirical risk functional defined over RKHS, the optimal solution can be expressed as

a linear combination of the kernel evaluations at the training points. In the context of CPAI, the objective is to find a discriminant vector $\theta \in \mathcal{H}$ that satisfies $(\Psi(x^i), \theta)_{\mathcal{H}} = \beta_i$. According to the Representer Theorem, the optimal θ must lie within the span of the mapped training samples:

$$\theta = \sum_{i=1}^n \alpha_i \Psi(x^i). \quad (6)$$

This theoretical foundation allows us to transform the problem from an infinite-dimensional optimization in $\mathcal{H}$ into a finite-dimensional linear system:

$$K\alpha = \beta, \quad (7)$$

where K is the kernel matrix defined by:

$$K = [\kappa(x^i, x^l)]_{i,l \in \{1, \dots, n\}}. \quad (8)$$

The existence and uniqueness of the solution of Eq. (7) depend on the nonsingularity of the kernel matrix K . By Mercer's theorem [35], any symmetric, continuous, positive semi-definite kernel admits an eigenfunction expansion with non-negative eigenvalues, so the corresponding Gram matrix K is positive semi-definite. We recall that a kernel function κ is said to be strictly positive definite if its corresponding kernel matrix K is positive definite for any set of distinct data points. In such a case, the system is guaranteed to admit a unique solution. More precisely, a sufficient condition for a symmetric matrix K to be strictly positive definite is that it satisfies the strictly diagonally dominant (SDD) property, defined as $|K_{ii}| > \sum_{l \neq i} |K_{il}|$ for all i [22]. Consider the RBF kernel, $\kappa(x, y) = \exp(-\|x - y\|^2 / (2\sigma^2))$, with a bandwidth parameter $\sigma > 0$. By construction, K is a symmetric matrix with unit diagonal elements ($K_{ii} = 1$) and positive off-diagonal entries ($0 < K_{il} < 1$ for distinct samples $x_i \neq x_l$).

As σ decreases (or the bandwidth $\gamma = 1/2\sigma^2$ increases), the off-diagonal terms $\kappa(x_i, x_l)$ exponentially decay toward zero. Consequently, for a sufficiently small σ , the row sum $\sum_{l \neq i} K_{il}$ becomes strictly less than 1, rendering the matrix K strictly diagonally dominant and thus strictly positive definite. Similarly, for the Laplacian kernel $\kappa(x, y) = \exp(-\alpha\|x - y\|)$, setting the parameter α large enough ensures that the SDD property is maintained.

While the theoretical positive definiteness of the kernel matrix K ensures invertibility, it can still become numerically ill-conditioned in practice (i.e., having eigenvalues approaching zero). This leads to numerical instability and potential class-merging singularities when solving the linear system. For instance, with the RBF kernel $\kappa(x, z) = \exp(-\gamma\|x - z\|^2)$, a very small bandwidth parameter γ causes the off-diagonal elements of K to approach 1, rendering the matrix nearly singular and sensitive to noise.

To address this instability and provide a structural safeguard against the singularity problem observed in NFST, a Tikhonov regularization term is introduced. This transforms the problem into a robust, regularized system, which is the

final formulation solved in our CPAI implementation:

$$(K + \lambda I)\alpha = \beta, \quad (9)$$

where $\lambda > 0$ is a small regularization hyperparameter and I is the identity matrix. From a spectral perspective, the addition of the diagonal term λI shifts all eigenvalues of K by exactly λ . Since the eigenvalues of a symmetric positive semi-definite kernel matrix are non-negative ($\lambda_j \geq 0$), the eigenvalues of $(K + \lambda I)$ are strictly positive ($\lambda_j + \lambda > 0$). This ensures that the system is always strictly positive definite and well-conditioned, guaranteeing a unique, stable solution for α even in the presence of redundant or non-distinct IoT data samples.

Hyperparameter selection. A key strength of CPAI is its robustness to hyperparameter choice: neither the regularization parameter λ nor the kernel-specific bandwidth parameters require dataset-dependent tuning. We adopt a single set of fixed values across all seven benchmark datasets, and empirical evidence (Appendix C) confirms that classification performance is stable under these choices across a wide operating range.

Regularization parameter λ . We adopt fixed default values: $\lambda = 10^{-9}$ for CPAI-OvA on the kernel path and $\lambda = 10^{-5}$ for CPAI-OvR. The smaller value for OvA reflects the smoother nature of its scalar integer-index target, which is less sensitive to small perturbations than OvR's binary $\{0, 1\}$ targets. A sensitivity analysis (Appendix C, Tables 10-11) confirms that these defaults lie in the flat region of the λ -MCC curve on every benchmark dataset.

Kernel hyperparameters. Because CPAI's discriminant construction does not prescribe a specific kernel, the choice of kernel family, rather than the kernel's bandwidth parameter, is the dominant factor in performance, as demonstrated by the kernel ablation in Section 5. For the bandwidth parameters themselves, we adopt fixed dataset-independent conventions. For γ -parameterised kernels (RBF, polynomial, sigmoid), we use the `scikit-learn` default $\gamma = 1/d_{\text{features}}$, where d_{features} is the number of input features. For the exponential-rate kernels (Laplacian, Abel) we use $\alpha = 0.1$, and for the Sobolev kernel we use Matérn-family order $k = 0.5$ with $d = 1$; these two conventions are inherited from the original CPAI implementation. All values are functions of the kernel or the feature count alone, and are therefore fixed without consulting any test data.

Remark 1. Theoretically, scaling the target values b_j scales the resulting coefficient vector α proportionally. However, we have observed that choosing excessively large values significantly increases the norm $\|\alpha\|$, which can amplify the model's sensitivity to data noise and numerical instability. More precisely, because the system $(K + \lambda I)\alpha = \beta$ is linear in β , scaling all targets by a factor c produces a coefficient vector $c\alpha$ and a projection $cf(x)$: decision boundaries scale identically and the classification outcome is invariant. Consequently, large spacing does not improve

separability, the discriminant geometry is scale-invariant, but a larger $\|\alpha\|$ amplifies floating-point rounding errors in $f(x)$ and increases sensitivity to input noise. Conversely, if two targets are close relative to noise, the projection variance bound $\text{Var}(y) \leq 1$ established in Section 4 (Eq. (16)) ensures a tight manifold collapse around each target, preventing class-manifold overlap. We recommend unit-spaced integer targets for OvA to maintain numerical stability and minimize computational overhead, and a binary strategy for OvR to achieve superior detection accuracy.

3.3. CPAI vs. Regularized Regression Frameworks

While algebraically identical to Ridge Regression in Dual Variables (also known as Kernel Ridge Regression, KRR) formally introduced by Saunders et al. [43], CPAI is technically distinguished by its discriminant optimization goal. In the existing literature, regression-based anomaly detections typically focus on (i) feature reconstruction, where models seek to predict one feature from others, flagging high residual errors as anomalies; or (ii) boundary estimation, where models like the One-Class Extreme Learning Machine (OC-ELM) [30] and the KOC+ framework [21] seek a continuous surface that wraps the legitimate data manifold. KRR minimizes a penalized prediction error with respect to given numerical labels, treating the target values as fixed regression outputs. In contrast, CPAI treats those same target values as geometric coordinates that are deliberately chosen to prevent the class-merging singularities inherent in prior null-space methods. Furthermore, as proven in Section 4, CPAI serves as a structural filter that aligns the transformation axis with the data centroid in the RKHS while nullifying local noise. This enables the model to identify anomalies through structural mismatch rather than mere numerical deviation, ensuring robustness against subtle attacks that mimic the statistical range of legitimate traffic.

3.4. Implementation of Kernel CPAI Algorithms

Based on the theoretical framework presented in Section 3.2, we detail the procedures for obtaining the discriminant coefficients for both OvA and One-vs-Rest (OvR) strategies. Unlike the linear case, the kernelized CPAI directly computes the coefficient vector α , which defines the discriminant direction implicitly in the RKHS.

Algorithm 2 outlines the One-vs-Rest (OvR) strategy. This approach constructs c separate classifiers to isolate each class from the remaining data manifold.

Decision Rule: At inference time, a test sample x is classified by projecting it into the discriminant space via the decision function $f(x) = \sum_{i=1}^n \alpha_i \kappa(x, x^i)$. The final decision depends on the training strategy.

- **OvA classification:** The scalar projection $f(x)$ is rounded to the nearest integer in $\{0, 1, \dots, c-1\}$:

$$\hat{y} = \text{round}(f(x)), \quad (11)$$

returning the class whose target $b_{\hat{y}} = \hat{y}$ is closest to $f(x)$. Values falling outside $[0, c-1]$ may be flagged as anomalies.

Algorithm 1: Kernel CPAI: One-vs-All (OvA)

Input: Q : Training data matrix; κ : Kernel function; γ, λ : Hyperparameters.

Output: α : Optimal coefficient vector.

- 1: **Construct** the kernel matrix $K \in \mathbb{R}^{n \times n}$ where $K_{il} = \kappa(x^i, x^l)$ using bandwidth γ .
- 2: **Initialize** the multi-class target vector:

$$\beta = (\underbrace{b_1, \dots, b_1}_{n_1}, \dots, \underbrace{b_j, \dots, b_j}_{n_j}, \dots, \underbrace{b_c, \dots, b_c}_{n_c})^T$$

- 3: Solve $(K + \lambda I)\alpha = \beta$ for α .
 - 4: **return** α .
-

Algorithm 2: Kernel CPAI: One-vs-Rest (OvR)

Input: Q : Training data matrix; κ, γ, λ .

Output: $\mathcal{A} = \{\alpha^1, \alpha^2, \dots, \alpha^c\}$: Set of coefficient vectors.

- 1: **Construct** the kernel matrix $K \in \mathbb{R}^{n \times n}$ using parameters γ .
- 2: **Initialize** $\mathcal{A} \leftarrow \emptyset$.
- 3: **for** $i = 1$ to c **do**
- 4: **Construct** the binary target vector for class i :

$$\beta^i = (\underbrace{0, \dots, 0}_{n_1}, \dots, \underbrace{1, \dots, 1}_{n_i}, \dots, \underbrace{0, \dots, 0}_{n_c})^T \quad (10)$$

- 5: Solve $(K + \lambda I)\alpha^i = \beta^i$ for α^i .
 - 6: Append α^i to $\mathcal{A}$.
 - 7: **end for**
 - 8: **return** $\mathcal{A}$.
-

- **OvR classification:** The sample is assigned to the class j whose discriminant function $f_j(x)$ yields the highest score:

$$\hat{y} = \arg \max_{j \in \{1, \dots, c\}} f_j(x). \quad (12)$$

- **Anomaly identification:** CPAI provides a natural mechanism for identifying anomalies. Since normal samples are forced to collapse toward a specific target b_{normal} , any test instance x whose projected value $f_{\text{normal}}(x)$ deviates significantly from this target is flagged as an anomaly, specifically if $f_{\text{normal}}(x) < \tau$, where τ is a confidence threshold. This structural deviation indicates that the sample does not conform to the learned manifold of legitimate network behavior.

3.5. Computational Complexity Analysis

While direct methods to solve the system (9), like Cholesky decomposition ($O(n^3)$), are practical for moderate datasets, we utilize the Conjugate Gradient (CG) method for large-scale scenarios.

Training Complexity. The training phase is divided into main stages: (i) constructing the kernel matrix requires $O(n^2d)$, and solving Eq.(9) via CG. Each iteration costs $O(n^2)$, resulting in a total cost of $O(Tn^2)$ for OvA and $O(n^2d + cTn^2)$ for OvR, where T is the number of iterations required for convergence. We note that this CG-based bound is the relevant cost model for large-scale settings; at the operating sizes used in our experiments, we use a direct Cholesky factorization whose empirical OvA-vs-OvR behavior is discussed in Section 5.6.

Estimation of Iterations (T). The number of iterations T required to reach precision ϵ is theoretically bounded by [48]

$$T \leq \frac{1}{2} \sqrt{\kappa} \ln \left(\frac{2}{\epsilon} \right), \quad (13)$$

where κ is the condition number defined as the ratio of the maximum to the minimum eigenvalue of the matrix $K + \lambda I$:

$$\kappa = \frac{\lambda_{\max}(K) + \lambda}{\lambda_{\min}(K) + \lambda}. \quad (14)$$

For kernels such as the RBF kernel (where $K_{ii} = 1$ for all $i = 1, \dots, n$), the numerator is bounded above by $n + \lambda$, while the denominator is bounded below by λ via Tikhonov regularization. Thus, the worst-case condition number is $\kappa \leq (n + \lambda)/\lambda \approx n/\lambda$, yielding a theoretical iteration bound of

$$T = O \left(\sqrt{\frac{n}{\lambda}} \right). \quad (15)$$

In practice, $T \ll n$ because the eigenvalues of kernel matrices typically decay rapidly or cluster [31, 54], allowing CG to converge much faster than the theoretical worst-case bound. This makes the $O(Tn^2)$ approach significantly more efficient than $O(n^3)$ direct solvers for large n .

Inference Complexity. Classifying a new instance requires evaluating the kernel vector in $O(nd)$. The total overhead scales as $O(nd)$ for OvA and $O(nd + cn)$ for OvR (computing the kernel vector once per test sample). This linear efficiency ensures that CPAI facilitates real-time, line-rate intrusion detection on resource-constrained IoT gateways.

4. Further Theoretical Analysis

To distinguish CPAI from generic regression frameworks and clarify its structural advantages over traditional null-space methods, we provide a deeper analysis of its geometric stability and spectral properties. For the sake of theoretical clarity, the analysis in this section focuses on the behavior of a single-class manifold, where the target vector is set as the all-ones vector ($\beta = 1$). In this context, any deviation of a new sample from this target indicates an anomaly.

4.1. Sample-size Invariant Stability

Let $y = [f(x^1), \dots, f(x^n)]^T = K\alpha$ be the outputs vector for the training samples. It follows from the CPAI solution that $y = 1 - \lambda\alpha$. The variance of the projected points is derived as:

$$\text{Var}(y) = \frac{\lambda^2}{n} \|\alpha - \bar{\alpha}1\|^2 \quad (16)$$

where $\bar{\alpha} = \frac{1}{n}1^T\alpha$. Furthermore, since $(K + \lambda I)$ is positive definite, the norm $\|\alpha\|$ is bounded by $\sqrt{n}/\lambda$. Consequently, $\text{Var}(y) \leq 1$. This bound holds regardless of the sample size (n), dimensionality (d), or the kernel eigenstructure.

4.2. Spectral Dynamics

Let the kernel matrix admit the eigendecomposition $K = \sum_{j=1}^n \lambda_j v_j v_j^T$, with $\{v_j\}$ forming an orthonormal basis. The CPAI solution admits the spectral expansion:

$$\alpha = \sum_{j=1}^n \frac{(1, v_j)}{\lambda_j + \lambda} v_j. \quad (17)$$

PROOF. To derive the spectral expansion of the CPAI coefficient vector α , we consider the regularized system defined in Eq. (9) with the unit target $\beta = 1$:

$$(K + \lambda I)\alpha = 1. \quad (18)$$

Based on the eigendecomposition of the kernel matrix $K = \sum_{j=1}^n \lambda_j v_j v_j^T$ and the identity $I = \sum_{j=1}^n v_j v_j^T$, the operator $(K + \lambda I)$ can be expressed in its spectral form as:

$$K + \lambda I = \sum_{j=1}^n (\lambda_j + \lambda) v_j v_j^T. \quad (19)$$

Since K is a symmetric positive semi-definite matrix ($\lambda_j \geq 0$) and $\lambda > 0$, the inverse of this operator is well-defined and computed by inverting the shifted eigenvalues:

$$(K + \lambda I)^{-1} = \sum_{j=1}^n \frac{1}{\lambda_j + \lambda} v_j v_j^T. \quad (20)$$

Applying this inverse to the target vector 1 yields:

$$\begin{aligned} \alpha &= (K + \lambda I)^{-1} 1 \\ &= \left(\sum_{j=1}^n \frac{1}{\lambda_j + \lambda} v_j v_j^T \right) 1 \\ &= \sum_{j=1}^n \frac{1}{\lambda_j + \lambda} v_j (v_j^T 1) \\ &= \sum_{j=1}^n \frac{(1, v_j)}{\lambda_j + \lambda} v_j. \end{aligned}$$

Notice that the term $S_j := (1, v_j)$ represents the projection of the data centroid onto the principal axes of the manifold in the RKHS.

4.2.1. Centroid-to-Axis Alignment Theorem

Recall $\mathcal{H}$ as the RKHS induced by the kernel $\kappa(\cdot, \cdot)$ and Ψ as the implicit feature map. The kernel mean element of the class is $\mu = \frac{1}{n} \sum_{k=1}^n \Psi(x^k)$. Analogous to Kernel Principal Component Analysis (KPCA) [46], the j^{th} principal axis $u_j \in \mathcal{H}$ is constructed as $u_j = \frac{1}{\sqrt{\lambda_j}} \sum_{i=1}^n v_{j,i} \Psi(x^i)$.

Theorem 3. *The projection of the centroid μ onto the j^{th} principal axis u_j in the RKHS is proportional to S_j :*

$$(\mu, u_j)_{\mathcal{H}} = \frac{\sqrt{\lambda_j}}{n} S_j. \quad (21)$$

PROOF. By substituting the definitions of μ and u_j :

$$\begin{aligned} (\mu, u_j)_{\mathcal{H}} &= \left(\frac{1}{n} \sum_{k=1}^n \Psi(x^k), \frac{1}{\sqrt{\lambda_j}} \sum_{i=1}^n v_{j,i} \Psi(x^i) \right)_{\mathcal{H}} \\ &= \frac{1}{n\sqrt{\lambda_j}} \sum_{k=1}^n (K v_j)_k. \end{aligned}$$

Since v_j is an eigenvector of K ($K v_j = \lambda_j v_j$), we have:

$$(\mu, u_j)_{\mathcal{H}} = \frac{\lambda_j}{n\sqrt{\lambda_j}} \sum_{k=1}^n v_{j,k} = \frac{\sqrt{\lambda_j}}{n} S_j. \quad (22)$$

4.2.2. Structural Implication

To clarify the physical orientation of the learned model, we examine the discriminant vector θ in the RKHS. By combining the spectral expansion with the construction of principal axes u_j , we derive the following:

$$\theta = \sum_{j=1}^n \left(\frac{\sqrt{\lambda_j} S_j}{\lambda_j + \lambda} \right) u_j. \quad (23)$$

This derivation provides a profound justification for the CPAI solution. It acts as a structural filter:

- $S_j \neq 0$: These axes are well-aligned with the data centroid μ , representing global invariant features. CPAI preserves these channels.
- $S_j \approx 0$: These axes represent local noise or sample-specific variations orthogonal to the centroid. CPAI effectively nullifies these directions.

4.3. Local Manifold Preservation

As illustrated in Fig. 3, CPAI transforms the input feature space into a topological landscape where normal data clusters form a *Plateau* at height b_j . A key theoretical property of this mapping is that $f(x)$ is globally bounded and strictly below the target b_j , which we prove as follows:

Theorem 4. *For any $\lambda > 0$ and a positive definite kernel κ , the CPAI prediction function $f(x)$ for a single class with target $b_j = 1$ satisfies $f(x) < 1$ for all $x \in \mathbb{R}^d$.*

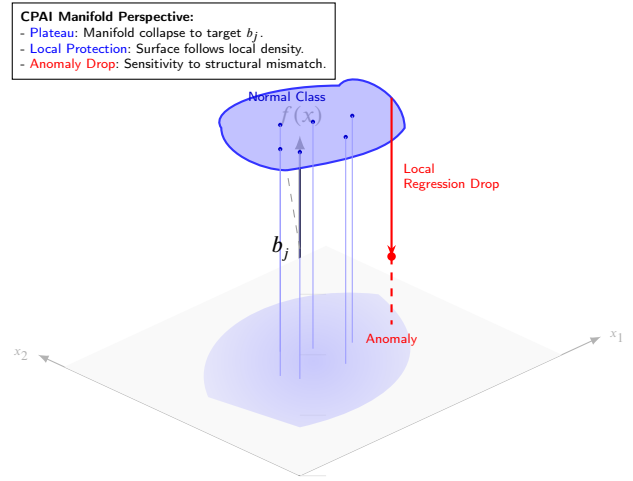


Figure 3: Geometric mechanism of CPAI: Normal samples are lifted to a target plateau at b_j , while a local anomaly suffers an altitude drop due to structural mismatch.

PROOF. Consider the CPAI system $(K + \lambda I)\alpha = 1$. Let $K = V\Lambda V^T$ be the eigendecomposition of the kernel matrix, where $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_n)$ with $\lambda_i \geq 0$. The prediction at any point x is given by $f(x) = \mathbf{k}(x)^T \alpha$, where $\mathbf{k}(x) = [\kappa(x, x_1), \dots, \kappa(x, x_n)]^T$. Substituting the solution for α :

$$f(x) = \mathbf{k}(x)^T V(\Lambda + \lambda I)^{-1} V^T \mathbf{1} = \sum_{j=1}^n \frac{(\mathbf{k}(x)^T v_j)(v_j^T \mathbf{1})}{\lambda_j + \lambda} \quad (24)$$

At a training point x_i , $\mathbf{k}(x_i)$ is the i -th column of K , thus $\mathbf{k}(x_i)^T v_j = \lambda_j v_{j,i}$. The prediction becomes:

$$f(x_i) = \sum_{j=1}^n \left(\frac{\lambda_j}{\lambda_j + \lambda} \right) v_{j,i} (v_j^T \mathbf{1}) \quad (25)$$

Since $\lambda > 0$, the scaling factor satisfies $\frac{\lambda_j}{\lambda_j + \lambda} < 1$ for all j . Furthermore, since V is an orthogonal matrix, $\sum_{j=1}^n v_{j,i} (v_j^T \mathbf{1}) = [V V^T \mathbf{1}]_i = [\mathbf{1}]_i = 1$. It follows that:

$$f(x_i) < \sum_{j=1}^n 1 \cdot v_{j,i} (v_j^T \mathbf{1}) = 1 \quad (26)$$

Because the kernel $\kappa(x, y)$ decays as x moves away from the training density, the maximum value of $f(x)$ is anchored at the training manifold. Therefore, $f(x) < 1$ holds globally.

5. Experiments

This section outlines our experiments designed to address the following research questions:

- **RQ1:** How effectively does the CPAI model classify attacks under two training approaches?
- **RQ2:** How significantly do the training approach and kernel selection impact the classification performance?

- **RQ3:** How does the performance of the CPAI model compare to baseline models?

5.1. Datasets

To assess the efficacy of our proposed algorithm, we conduct a comprehensive evaluation across seven widely used IoT intrusion detection datasets: BoT-IoT, IoTID20, ToN-IoT, N-BaIoT, CIC-IoT2023, Edge-IIoTset, and 5G-NIDD. These datasets are specifically designed to emulate the complexities of IoT network environments and encompass various attack traffic patterns. BoT-IoT, IoTID20, ToN-IoT, N-BaIoT and CIC-IoT2023 are widely adopted IoT IDS benchmarks; Edge-IIoTset and 5G-NIDD extend the evaluation to Industrial-IoT and 5G-edge scenarios, respectively, and contribute the two highest class counts in the collection (fifteen and nine), providing a stronger test of the proposed multi-class framework. A concise overview of the utilized datasets is provided below:

BoT-IoT [29]: The BoT-IoT dataset, published by the Cyber Range Lab of UNSW Canberra, serves as an important resource for supporting research in IoT network security. This dataset comprises various network traffic data captured from IoT devices infected with diverse botnet malware strains. These malicious programs are known to initiate a wide spectrum of cyberattacks, encompassing distributed denial of service (DDoS), brute-force incursions, and reconnaissance scans. The network traffic data acquisition took place within a meticulously controlled environment that replicates real-world IoT network configurations. Through comprehensive analysis, a comprehensive dataset of over 20 features was extracted from the raw network traffic, offering insights into key network parameters, flow dynamics, and traffic volume.

IoTID20 [53]: The IoTID20 dataset is well-designed to facilitate the development of IDSs for the IoT environment. It simulates real-world network traffic in an IoT setup and provides a foundation for identifying and mitigating malicious activities. IoTID20 covers a wide range of cyberattacks that specifically target IoT devices and networks. These include Distributed Denial of Service (DDoS), man-in-the-middle (MITM), data exfiltration, and scanning attacks.

ToN-IoT [4]: The ToN-IoT dataset, provided by the Cyber Range Lab of UNSW Canberra, is a resource designed to facilitate the development of artificial intelligence-driven security solutions tailored for industrial IoT networks. Unlike the BoT-IoT dataset, which focuses on attack traffic, the ToN-IoT dataset primarily encompasses benign network traffic generated by legitimate IoT devices operating under normal, non-compromised conditions. Furthermore, the ToN-IoT dataset also provides diversity, encompassing a wide range of IoT applications and communication protocols commonly deployed in real-world IIoT environments.

N-BaIoT [34]: The N-BaIoT dataset is a valuable resource for analysing the normal behavioural patterns of different IoT devices, encompassing sensors, actuators, and smart home appliances. By capturing the network traffic generated by these devices under benign conditions, the dataset establishes a baseline for comparison, aiding in identifying

Table 2

Description of the seven benchmark datasets: total number of preprocessed records, feature dimensionality, number of classes, training partition size after the noise filter, and held-out test partition size.

Dataset	Total samples	Features	Classes	n_{train}	n_{test}
BoT-IoT	10,118	23	6	7,729	2,024
IoTID20	10,000	79	5	7,680	2,000
ToN-IoT	9,597	41	8	7,333	1,920
N-BaIoT	9,000	115	9	6,880	1,800
CIC-IoT2023	10,000	48	6	7,640	2,000
Edge-IIoTset	15,000	42	15	11,440	3,000
5G-NIDD	9,000	48	9	6,880	1,800

malicious activity. Moreover, the N-BaIoT dataset is notable for covering a wide range of IoT applications and communication protocols. This diversity ensures that the dataset is representative of real-world IoT deployments, enhancing its relevance and applicability to security research. The N-BaIoT dataset also includes instances of botnet attacks originating from two prominent botnet families: Mirai and Bashlite. These attacks provide valuable insights into the tactics and techniques employed by malicious actors seeking to exploit vulnerabilities in IoT devices.

CIC-IoT2023 [37]: The CIC-IoT2023 dataset, published by the Canadian Institute for Cybersecurity (CIC), represents a comprehensive resource designed to evaluate the performance and effectiveness of IDSs specifically tailored for IoT environments. This dataset includes a diverse range of IoT devices and simulates many cyberattacks that pose significant threats to IoT ecosystems. For the aim of this research, the attacks contained within the CIC-IoT2023 dataset have been systematically categorized into five distinct attack categories: DDoS/DoS, scan, web attacks, spoofing, and botnet attacks.

Edge-IIoTset [18]: Edge-IIoTset is a recent large-scale Industrial-IoT IDS benchmark released by Ferrag et al. in 2022. It covers fourteen attack types across five categories (DoS/DDoS, information-gathering, man-in-the-middle, injection, and malware) plus benign traffic, for a total of fifteen classes. Traffic was captured on a purpose-built seven-layer IIoT testbed with more than 10 device types, reflecting the heterogeneous device population found in real IIoT networks.

5G-NIDD [42]: 5G-NIDD is a 5G-focused IDS dataset released by Samarakoon et al. in 2022, generated on a functional 5G testbed comprising gNodeB equipment, 5G-core functions, and real user-equipment traffic. It contains nine attack classes plus benign traffic and captures cellular-edge and 5G-core signaling traffic in addition to conventional IP flows, covering the radio-access and control-plane protocols relevant to large-scale 5G-IoT deployments.

Experimental Protocol and Reproducibility. Every dataset is partitioned using a stratified 80/20 hold-out split. The results reported in Section 5.5 are the mean $\pm$ standard deviation over five seeds. The preprocessing pipeline is

identical across all methods in the comparison: identifier fields are removed, numerical features are standardized with a scikit-learn scaler depending on each dataset, and the LOF-based noise filter described above is applied only to the training partition. No feature selection or dimensionality reduction is applied. Categorical fields that survive the identifier drop are label-encoded. Rows with $\pm\infty$ or NaN values are either dropped or imputed with the column mean, depending on the dataset. The test partition is never inspected during model selection. Table 2 summarises the seven benchmarks, with the values of n_{train} in the table reflecting the training size after this filter has been applied.

Class imbalance is addressed at the evaluation-design stage: every benchmark is first reduced to a uniform per-class sample count, and the same balanced subsample is used for CPAI and all baselines. This scope is deliberately a controlled cross-method comparison, not a deployment-realism study under natural traffic frequencies, and Section 6.3 explicitly flags extreme natural-frequency imbalance as a regime in which CPAI degrades, which we leave for future work.

A full reproducibility checklist, such as random seeds, library versions, and hardware configuration, is provided in Section 5.6 (Experimental protocol paragraph). A reference implementation of CPAI (linear and kernel variants, both OvA and OvR algorithms, the hyperparameter path, the λ sensitivity harness, and the seven-dataset preprocessing pipeline) is available at ¹

5.2. Evaluation Metrics

To evaluate the detection performance of our proposed algorithm in classifying cyber attacks, we use a comprehensive set of benchmarking metrics calculated from the confusion matrix. These include Accuracy (ACC), True Positive Rate (TPR), Positive Predictive Value (PPV), and False Positive Rate (FPR) [49], which are commonly used to evaluate classifier performance. While these metrics provide an overall assessment of correctness, their reliability can be compromised in imbalanced datasets. To address this limitation, the F1-score [40], which balances precision and recall, and the Matthews Correlation Coefficient (MCC) [33, 13], which considers all four confusion matrix components, are incorporated to provide a more robust evaluation. We describe the formula for each metric below:

- Accuracy (ACC):

$$ACC = \frac{TP + TN}{TP + TN + FP + FN}$$

- Recall - True Positive Rate (TPR):

$$TPR = \frac{TP}{TP + FN}$$

- False Alarm Rate - False Positive Rate (FPR):

$$FPR = \frac{FP}{FP + TN}$$

- Precision - Positive Predictive Value (PPV):

$$PPV = \frac{TP}{TP + FP}$$

- F1-score:

$$F1\text{-score} = \frac{2 * TPR * PPV}{TPR + PPV}$$

- Matthews Correlation Coefficient (MCC):

$$MCC = \frac{TP \times TN - FP \times FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}$$

In addition to the six confusion-matrix metrics listed above, we report two ranking-based metrics [17] that characterise score-level discrimination independently of the classification threshold:

- One-vs-Rest macro AUROC [24]: for a C -class problem, the classifier produces, for every sample, a score vector $\hat{p} = (\hat{p}_1, \dots, \hat{p}_C)$. Treating each class c in turn as the positive class and the remaining $C - 1$ classes as the negative class yields C binary ROC curves; the one-vs-rest macro AUROC is their unweighted mean:

$$AUROC_{\text{OvR}} = \frac{1}{C} \sum_{c=1}^C AUC(\hat{p}_c, \mathbb{I}[y = c]),$$

where $\mathbb{I}[\cdot]$ is the indicator function. The macro average weights every class equally and is therefore sensitive to minority-class behaviour, a desirable property in the imbalanced intrusion-detection setting.

- Binary attack-vs-benign AUROC: the benign class is treated as negative and every attack class as positive. The per-sample attack score is $s = 1 - \hat{p}_{\text{benign}}$, and the binary AUROC is computed from $(s, \mathbb{I}[y \neq \text{benign}])$. This quantifies how well the model separates any attack from normal traffic without committing to a specific attack category - a direct proxy for deployed IDS alerting behavior.

5.3. Baseline Competative Models

In this research, we employ a comprehensive set of baseline algorithms to evaluate our proposed approach. These baselines span two methodological families. The classical discriminant and distance-based family comprises Gaussian Naive Bayes (GauNB) [27], Kernel Null Foley-Sammon Transformation (KNFST) [9], K-Nearest-Neighbor (KNN) [39], Linear Discriminant Analysis (LDA) [19], Nearest Centroid (NC) [52], Nu-SVM (NuSVC) [47], Radius Neighbors Classifier (RNC) [7], and Stochastic Gradient Descent SVM (SGD) [55] - a linear SVM trained with stochastic gradient descent, implemented via scikit-learn's `SGDClassifier` with `loss='hinge'`. The modern lightweight-IDS family adds four baselines used in recent IoT-IDS literature: LightGBM [28], a multilayer perceptron (MLP),

¹<https://anonymous.4open.science/r/cpai-classifier-0F48/README.md>.

Table 3

Performance evaluation of CPAI-OvA across the first four benchmark datasets: ACC, TPR, and FPR reported as mean $\pm$ standard deviation over five seeds. The best score per column is in red, and the second-best is in blue.

Kernel	BoT-IoT			IoTID20			ToN-IoT			N-BaIoT		
	ACC	TPR	FPR	ACC	TPR	FPR	ACC	TPR	FPR	ACC	TPR	FPR
None	99.68 $\pm$ 0.09	84.56 $\pm$ 0.26	0.189 $\pm$ 0.050	82.42 $\pm$ 0.17	39.38 $\pm$ 0.42	10.550 $\pm$ 0.101	83.88 $\pm$ 0.32	19.36 $\pm$ 1.66	9.070 $\pm$ 0.182	87.55 $\pm$ 0.24	33.99 $\pm$ 1.06	6.915 $\pm$ 0.131
Linear	99.70 $\pm$ 0.04	84.62 $\pm$ 0.11	0.178 $\pm$ 0.024	82.46 $\pm$ 0.18	39.49 $\pm$ 0.45	10.522 $\pm$ 0.107	83.59 $\pm$ 0.22	19.63 $\pm$ 0.85	9.230 $\pm$ 0.122	86.92 $\pm$ 0.08	31.16 $\pm$ 0.36	7.264 $\pm$ 0.044
Poly	99.71 $\pm$ 0.06	84.80 $\pm$ 0.15	0.168 $\pm$ 0.034	88.98 $\pm$ 0.31	55.77 $\pm$ 0.78	6.614 $\pm$ 0.186	92.43 $\pm$ 0.26	54.36 $\pm$ 0.89	4.258 $\pm$ 0.147	98.28 $\pm$ 0.06	82.25 $\pm$ 0.26	0.957 $\pm$ 0.032
Rbf	99.65 $\pm$ 0.11	98.89 $\pm$ 0.36	0.211 $\pm$ 0.064	89.46 $\pm$ 0.24	56.98 $\pm$ 0.59	6.324 $\pm$ 0.141	96.72 $\pm$ 0.21	75.16 $\pm$ 0.88	1.845 $\pm$ 0.118	99.01 $\pm$ 0.12	85.56 $\pm$ 0.54	0.548 $\pm$ 0.067
Sigmoid	97.07 $\pm$ 0.54	74.65 $\pm$ 1.97	1.708 $\pm$ 0.317	82.80 $\pm$ 1.44	40.33 $\pm$ 3.59	10.320 $\pm$ 0.863	88.25 $\pm$ 1.21	40.98 $\pm$ 4.25	6.609 $\pm$ 0.681	91.78 $\pm$ 1.66	52.99 $\pm$ 7.46	4.569 $\pm$ 0.921
Abel	99.68 $\pm$ 0.11	98.96 $\pm$ 0.35	0.192 $\pm$ 0.063	95.48 $\pm$ 0.56	78.71$\pm$7.79	2.762 $\pm$ 0.406	98.81 $\pm$ 0.15	88.23$\pm$5.66	0.674 $\pm$ 0.088	98.79 $\pm$ 0.12	90.54$\pm$5.00	0.679 $\pm$ 0.071
Laplacian	99.96$\pm$0.04	99.86$\pm$0.09	0.024$\pm$0.022	97.43$\pm$0.20	76.90 $\pm$ 0.49	1.544$\pm$0.118	99.25$\pm$0.05	85.59 $\pm$ 0.31	0.424$\pm$0.031	99.50$\pm$0.11	87.75 $\pm$ 0.51	0.278$\pm$0.062
Sobolev	97.79 $\pm$ 0.23	92.43 $\pm$ 0.65	1.326 $\pm$ 0.137	89.60 $\pm$ 0.26	74.00 $\pm$ 0.65	6.500 $\pm$ 0.164	94.79 $\pm$ 0.09	76.69 $\pm$ 0.47	2.978 $\pm$ 0.051	92.67 $\pm$ 0.16	67.01 $\pm$ 0.72	4.124 $\pm$ 0.090

Table 4

Performance evaluation of CPAI-OvA across the remaining three benchmark datasets: ACC, TPR, and FPR reported as mean $\pm$ standard deviation over five seeds. The best score per column is in red, and the second-best is in blue.

Kernel	CIC-IoT2023			Edge-IIoTset			5G-NIDD		
	ACC	TPR	FPR	ACC	TPR	FPR	ACC	TPR	FPR
None	95.49 $\pm$ 1.01	68.28 $\pm$ 3.61	2.628 $\pm$ 0.590	90.95 $\pm$ 0.02	25.89 $\pm$ 0.18	4.826 $\pm$ 0.013	88.59 $\pm$ 0.23	38.65 $\pm$ 1.03	6.340 $\pm$ 0.128
Linear	95.35 $\pm$ 1.07	68.09 $\pm$ 3.28	2.710 $\pm$ 0.623	89.56 $\pm$ 0.52	17.94 $\pm$ 1.97	5.579 $\pm$ 0.291	88.78 $\pm$ 0.33	39.51 $\pm$ 1.48	6.233 $\pm$ 0.183
Poly	99.79 $\pm$ 0.04	84.90 $\pm$ 0.12	0.120 $\pm$ 0.023	92.37 $\pm$ 0.08	36.54 $\pm$ 0.61	4.068 $\pm$ 0.043	97.67 $\pm$ 0.13	79.51 $\pm$ 0.59	1.295 $\pm$ 0.073
Rbf	99.84 $\pm$ 0.02	85.06 $\pm$ 0.09	0.095 $\pm$ 0.013	94.40 $\pm$ 0.08	51.76 $\pm$ 0.60	2.986 $\pm$ 0.043	98.06 $\pm$ 0.12	81.26 $\pm$ 0.55	1.079 $\pm$ 0.067
Sigmoid	97.38 $\pm$ 2.74	73.38 $\pm$ 13.34	1.528 $\pm$ 1.596	91.81 $\pm$ 0.12	32.35 $\pm$ 0.88	4.366 $\pm$ 0.063	95.51 $\pm$ 0.75	69.81 $\pm$ 3.38	2.493 $\pm$ 0.417
Abel	99.75 $\pm$ 0.03	98.93 $\pm$ 0.17	0.152 $\pm$ 0.019	95.61$\pm$0.07	60.81$\pm$0.52	2.342$\pm$0.037	99.39 $\pm$ 0.10	93.24 $\pm$ 5.11	0.344 $\pm$ 0.060
Laplacian	99.91$\pm$0.03	99.60$\pm$0.17	0.054$\pm$0.017	95.10 $\pm$ 0.13	56.99 $\pm$ 0.95	2.614 $\pm$ 0.068	99.40$\pm$0.08	95.28$\pm$4.22	0.339$\pm$0.045
Sobolev	96.39 $\pm$ 0.12	82.52 $\pm$ 0.64	2.166 $\pm$ 0.074	90.64 $\pm$ 0.08	29.81 $\pm$ 0.59	5.014 $\pm$ 0.042	95.16 $\pm$ 0.26	78.21 $\pm$ 1.16	2.724 $\pm$ 0.145

TabNet [5], and SAINT [50]. LightGBM is included separately: rather than a representative-based model, it is widely regarded as one of the strongest tabular classifiers on IoT-IDS benchmarks [6], and we include it to test whether CPAI remains competitive against a production-grade ensemble baseline despite relying solely on a closed-form kernel solution. For implementation, we utilize the scikit-learn Python library for the classical baselines, lightgbm for LGBM, pytorch-tabnet for TabNet, and our own reference implementation of SAINT in PyTorch. For KNFST, we provide our own implementation based on the original algorithm [9]. All baselines are trained with the same stratified 80/20 split, seed set, and preprocessing pipeline as CPAI, and use default library hyperparameters.

5.4. Multiclass Classification Performance Under Two Training Approaches With Different Kernel

This subsection addresses **RQ1** and **RQ2** by evaluating how kernel choice interacts with the two training strategies. Tables 3–4 report the performance of CPAI-OvA, and Tables 5–6 report the performance of CPAI-OvR, each on all seven benchmark datasets. To make the mean $\pm$ standard-deviation cells fit comfortably, each variant is split into two sub-tables: the first four datasets (BoT-IoT, IoTID20, ToN-IoT, N-BaIoT) and the remaining three (CIC-IoT2023, Edge-IIoTset, 5G-NIDD). For every (dataset, kernel) cell, the (scaler, poly) configuration with the highest mean MCC is selected; ACC, TPR macro, and FPR are then aggregated as mean $\pm$ standard deviation across the five seeds. Figures 4 and 5 summarise the cross-dataset aggregate in terms of mean MCC and mean F_1 -score.

5.4.1. The classification performance using different kernels

Tables 3 and 4 show that under the CPAI-OvA strategy the Laplacian kernel reaches the highest ACC on six of the seven datasets (BoT-IoT, IoTID20, ToN-IoT, N-BaIoT, CIC-IoT2023, and 5G-NIDD), while Abel wins on the fifteen-class Edge-IIoTset benchmark. The TPR ranking differs from the ACC ranking: Abel achieves the highest TPR across four datasets (IoTID20, ToN-IoT, N-BaIoT, Edge-IIoTset), whereas Laplacian achieves the highest TPR on BoT-IoT, CIC-IoT2023, and 5G-NIDD. This split between the best ACC kernel and the best TPR kernel indicates that Laplacian’s superiority on ACC sometimes comes from favoring majority classes under the OvA single-scalar discriminant, while Abel preserves minority-class recall at a modest ACC cost. The Laplacian kernel also dominates FPR on the same six datasets, where it leads ACC. The standard deviations of the top kernels are below 1 percentage point across all ACC cells and below 2 on almost all TPR cells, indicating stable across-seed behavior. Non-positive-definite kernels (None, Linear, Sigmoid) yield markedly weaker ACC and TPR together with higher FPR on the four harder datasets (IoTID20, ToN-IoT, N-BaIoT, Edge-IIoTset), consistent with the OvA discriminant’s reliance on a well-structured RKHS geometry.

Tables 5 and 6 show that the Laplacian kernel dominates CPAI-OvR on every dataset and every metric, reaching a perfect 100.00 $\pm$ 0.00 across ACC, TPR, and FPR on BoT-IoT and staying above 98.55 % ACC with TPR $\geq$ 89.13 % on the most challenging Edge-IIoTset benchmark. Abel is the second-best kernel on six of the seven datasets for all three metrics simultaneously; RBF replaces Abel in the second position on CIC-IoT2023, and on BoT-IoT the near-saturation of the problem places the no-kernel (None) row

Table 5

Performance evaluation of CPAI-OvR across the first four benchmark datasets: ACC, TPR, and FPR reported as mean $\pm$ standard deviation over five seeds. The best score per column is in red, and the second-best is in blue.

Kernel	BoT-IoT			IoTID20			ToN-IoT			N-BaloT		
	ACC	TPR	FPR	ACC	TPR	FPR	ACC	TPR	FPR	ACC	TPR	FPR
None	99.92 $\pm$ 0.03	99.76 $\pm$ 0.09	0.047 $\pm$ 0.019	90.96 $\pm$ 0.28	77.39 $\pm$ 0.69	5.652 $\pm$ 0.173	95.89 $\pm$ 0.06	76.69 $\pm$ 0.14	2.348 $\pm$ 0.034	97.04 $\pm$ 0.11	86.67 $\pm$ 0.52	1.667 $\pm$ 0.065
Linear	99.92 $\pm$ 0.04	99.74 $\pm$ 0.15	0.049 $\pm$ 0.026	90.96 $\pm$ 0.28	77.40 $\pm$ 0.70	5.650 $\pm$ 0.176	95.89 $\pm$ 0.06	76.69 $\pm$ 0.14	2.348 $\pm$ 0.034	97.04 $\pm$ 0.11	86.67 $\pm$ 0.52	1.667 $\pm$ 0.065
Poly	99.83 $\pm$ 0.05	99.39 $\pm$ 0.24	0.101 $\pm$ 0.027	93.38 $\pm$ 0.39	83.46 $\pm$ 0.97	4.135 $\pm$ 0.242	98.83 $\pm$ 0.17	95.05 $\pm$ 0.74	0.667 $\pm$ 0.098	99.42 $\pm$ 0.17	97.40 $\pm$ 0.77	0.325 $\pm$ 0.097
Rbf	99.83 $\pm$ 0.04	99.55 $\pm$ 0.10	0.103 $\pm$ 0.022	94.29 $\pm$ 0.22	85.72 $\pm$ 0.54	3.570 $\pm$ 0.136	99.23 $\pm$ 0.12	96.71 $\pm$ 0.58	0.438 $\pm$ 0.070	99.72 $\pm$ 0.15	98.76 $\pm$ 0.67	0.156 $\pm$ 0.084
Sigmoid	99.47 $\pm$ 0.15	98.47 $\pm$ 0.53	0.316 $\pm$ 0.088	85.25 $\pm$ 4.53	63.12 $\pm$ 11.31	9.220 $\pm$ 2.828	95.85 $\pm$ 2.87	82.35 $\pm$ 11.53	2.371 $\pm$ 1.638	94.06 $\pm$ 2.42	73.28 $\pm$ 10.91	3.340 $\pm$ 1.364
Abel	99.87 $\pm$ 0.04	99.68 $\pm$ 0.11	0.075 $\pm$ 0.026	95.99 $\pm$ 0.21	89.98 $\pm$ 0.53	2.505 $\pm$ 0.134	99.42 $\pm$ 0.08	97.41 $\pm$ 0.28	0.332 $\pm$ 0.044	99.86 $\pm$ 0.08	99.36 $\pm$ 0.35	0.081 $\pm$ 0.044
Laplacian	100.00$\pm$0.00	100.00$\pm$0.00	0.000$\pm$0.000	98.15$\pm$0.15	95.37$\pm$0.38	1.158$\pm$0.095	99.80$\pm$0.04	99.09$\pm$0.16	0.112$\pm$0.023	99.88$\pm$0.05	99.48$\pm$0.24	0.065$\pm$0.030
Sobolev	99.78 $\pm$ 0.05	99.36 $\pm$ 0.15	0.134 $\pm$ 0.028	95.38 $\pm$ 0.08	88.44 $\pm$ 0.20	2.890 $\pm$ 0.050	99.25 $\pm$ 0.05	96.86 $\pm$ 0.34	0.430 $\pm$ 0.030	99.55 $\pm$ 0.08	97.97 $\pm$ 0.38	0.254 $\pm$ 0.048

Table 6

Performance evaluation of CPAI-OvR across the remaining three benchmark datasets: ACC, TPR, and FPR reported as mean $\pm$ standard deviation over five seeds. The best score per column is in red, and the second-best is in blue.

Kernel	CIC-IoT2023			Edge-IIoTset			5G-NIDD		
	ACC	TPR	FPR	ACC	TPR	FPR	ACC	TPR	FPR
None	98.52 $\pm$ 0.41	92.76 $\pm$ 1.97	0.890 $\pm$ 0.248	94.22 $\pm$ 0.08	56.67 $\pm$ 0.58	3.095 $\pm$ 0.041	98.49 $\pm$ 0.13	93.20 $\pm$ 0.58	0.850 $\pm$ 0.073
Linear	98.71 $\pm$ 0.23	93.81 $\pm$ 0.93	0.772 $\pm$ 0.141	94.23 $\pm$ 0.09	56.71 $\pm$ 0.69	3.092 $\pm$ 0.049	98.49 $\pm$ 0.13	93.20 $\pm$ 0.58	0.850 $\pm$ 0.073
Poly	99.93 $\pm$ 0.04	99.80 $\pm$ 0.19	0.040 $\pm$ 0.025	97.21 $\pm$ 0.10	79.05 $\pm$ 0.78	1.497 $\pm$ 0.055	99.68 $\pm$ 0.05	98.57 $\pm$ 0.23	0.179 $\pm$ 0.029
Rbf	99.94 $\pm$ 0.04	99.81 $\pm$ 0.19	0.036 $\pm$ 0.024	97.52 $\pm$ 0.10	81.37 $\pm$ 0.78	1.331 $\pm$ 0.056	99.77 $\pm$ 0.04	98.94 $\pm$ 0.16	0.132 $\pm$ 0.020
Sigmoid	98.73 $\pm$ 1.20	94.10 $\pm$ 5.87	0.762 $\pm$ 0.723	94.38 $\pm$ 0.32	57.87 $\pm$ 2.43	3.010 $\pm$ 0.174	96.92 $\pm$ 1.12	86.13 $\pm$ 5.06	1.733 $\pm$ 0.632
Abel	99.90 $\pm$ 0.07	99.59 $\pm$ 0.36	0.060 $\pm$ 0.044	97.55 $\pm$ 0.03	81.65 $\pm$ 0.21	1.311 $\pm$ 0.015	99.79 $\pm$ 0.04	99.04 $\pm$ 0.18	0.119 $\pm$ 0.023
Laplacian	99.99$\pm$0.01	99.95$\pm$0.07	0.006$\pm$0.009	98.55$\pm$0.04	89.13$\pm$0.33	0.776$\pm$0.023	99.90$\pm$0.02	99.56$\pm$0.08	0.056$\pm$0.010
Sobolev	99.74 $\pm$ 0.05	98.81 $\pm$ 0.24	0.156 $\pm$ 0.027	97.13 $\pm$ 0.05	78.47 $\pm$ 0.40	1.538 $\pm$ 0.029	99.70 $\pm$ 0.02	98.66 $\pm$ 0.10	0.168 $\pm$ 0.012

as the runner-up. The OvR framework also substantially narrows the gap between positive-definite and non-positive-definite kernels relative to OvA: on Edge-IIoTset, for example, the ACC gap between Laplacian and Linear shrinks from 5.54 pp under OvA (Table 4) to 4.32 pp under OvR (Table 6), and the corresponding TPR gap shrinks from 39.05 pp to 32.42 pp. This reflects OvR’s ability to decompose a failing multiclass discriminant into a collection of easier binary subproblems.

5.4.2. The classification performance comparison between two training approaches

Aggregating Tables 3-6 across the seven benchmarks (Figures 4 and 5) confirms two systematic effects. First, CPAI-OvR dominates CPAI-OvA on every kernel: the mean-MCC gap is widest on the non-positive-definite kernels (None, Linear, Sigmoid: +0.37, +0.39, and +0.18 MCC, respectively) and narrows to +0.05 on Abel and +0.06 on Laplacian, where both variants operate in a well-structured RKHS. Second, within each variant, the kernel ranking is consistent: Laplacian achieves the best mean MCC (OvA: 0.91, OvR: 0.97), followed closely by Abel and RBF; the non-positive-definite kernels trail and exhibit the largest cross-dataset standard deviations.

5.5. Baseline Comparison Results

This subsection addresses **RQ3** by comparing CPAI against twelve baselines across all seven benchmark datasets. The baselines span two families introduced in Section 5.3: eight classical discriminant and distance-based methods (KNN, LDA, NuSVC, SGD, GauNB, NC, RNC, KNFST and four modern lightweight-IDS baselines (LightGBM, MLP, TabNet, SAINT). To facilitate interpretation, we highlight the highest-performing algorithm for each metric in red and

the second-best in blue. The analysis demonstrates that our proposed algorithm consistently outperforms the baseline approaches across multiple metrics.

Table 7 consolidates the results into a single cross-dataset summary: each cell is the unweighted mean of the per-dataset 5-seed mean, aggregated across the seven benchmarks. The p_t and p_w columns report paired two-sided t -test and Wilcoxon signed-rank p -values, respectively, for the null hypothesis that CPAI-OvR and the baseline produce the same per-dataset mean MCC ($n = 7$ paired samples, $p < 0.05$ indicates a statistically significant difference). Per-dataset detail tables with mean $\pm$ std and five-seed paired statistics are provided in Appendix E.

Real-world case study: DDoS-HTTP on Edge-IIoTset. To ground the aggregate numbers in a concrete deployment scenario, we trace one end-to-end use of the trained model. The Edge-IIoTset benchmark reflects a realistic industrial-IoT gateway: fifteen attack classes recorded on a seven-layer testbed with ten-plus heterogeneous device types. A new flow is preprocessed with the pipeline of Section 5.1, projected through the trained CPAI-OvR discriminant $f_j(x) = \sum_i \alpha_i^j \kappa(x, x^i)$ for every class j , and assigned to $\arg \max_j f_j(x)$. The confusion matrix in Appendix E (Figure 8) shows that 154/200 DDoS-HTTP flows are correctly labeled, and crucially for IDS deployment, zero DDoS-HTTP flows are misclassified as benign. Residual errors (fifteen to SQL injection, eleven to XSS, nine to Password) are cross-attack confusions within the web-attack family rather than missed detections, so a downstream security-response workflow would still raise an alert and present the analyst with a short candidate list rather than silently dropping the flow.

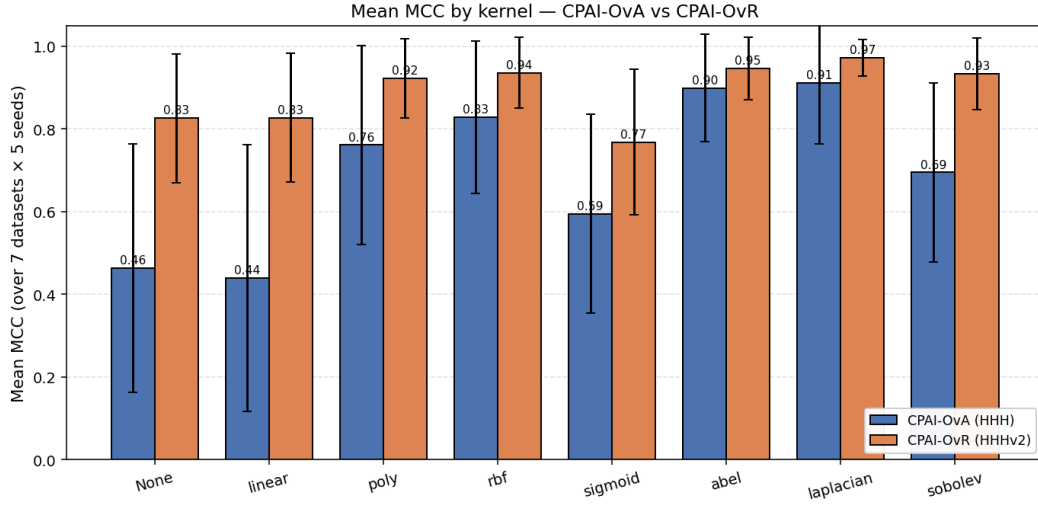


Figure 4: Comparative analysis of mean MCC score using different kernel functions between the two CPAI training strategies. For each (dataset, model, kernel) cell the best (scalar, poly) configuration is selected by mean MCC over five seeds; each bar is the unweighted average across the seven benchmark datasets, and error bars denote the cross-dataset standard deviation.

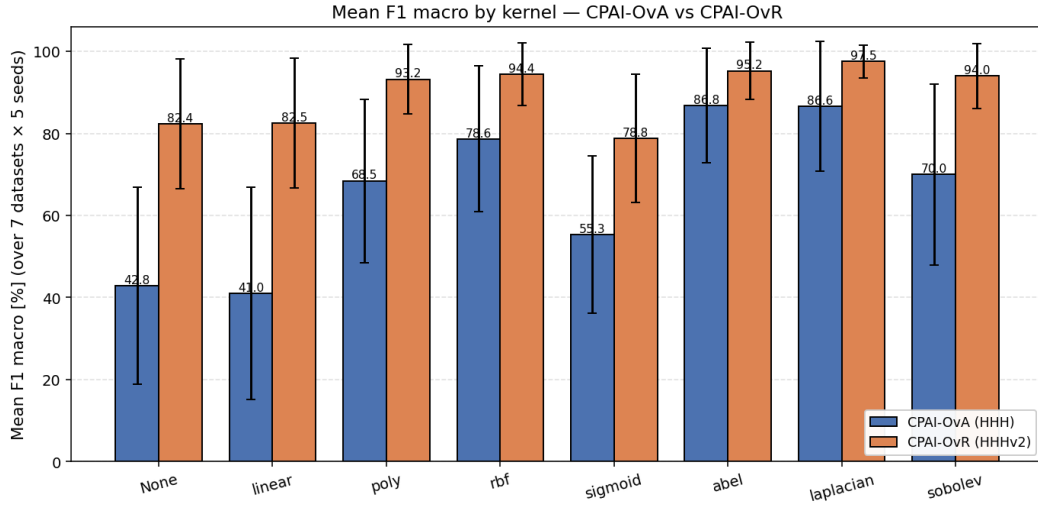


Figure 5: Comparative analysis of mean F_1 -score using different kernel functions between the two CPAI training strategies. Aggregation rule and error-bar definition are as in Figure 4.

In conclusion, CPAI-OvR is uniformly competitive across every dataset in our seven-benchmark roster: its MCC is either the best or within 0.005 of the best method on each individual dataset (Appendix E, Tables 12-18), and it is the top performer on four of the seven benchmarks (BoT-IoT, IoTID20, ToN-IoT, 5G-NIDD) while finishing a close second to LGBM on the remaining three (N-BaIoT, CIC-IoT2023, Edge-IIoTset). Additionally, on the cross-dataset summary (Table 7) CPAI-OvR ranks first on every one of the six metrics, narrowly ahead of LGBM (MCC 0.9722 vs. 0.9718; FPR 0.31% vs. 0.33%) and is statistically tied with LGBM on MCC ($p_w = 0.469$), while significantly outperforming every classical baseline and the modern tabular models MLP, TabNet, and SAINT ($p_w \leq 0.031$ against each).

5.6. Computational Efficiency and IoT Deployability

IoT intrusion-detection systems are commonly deployed on resource-constrained gateways rather than back-office servers, so wall-clock training cost, inference latency, and memory footprint are as relevant as detection accuracy. This subsection reports all three for the proposed CPAI variants and the baselines studied in Section 5.5.

Experimental protocol. For every (dataset, method, seed) triple we record the wall-clock time of `model.fit(X_train, y_train)` and of `model.predict + model.predict_proba`, excluding preprocessing. Peak resident set size (RSS) is recorded via `resource.getrusage(RUSAGE_SELF).ru_maxrss`. Experiments ran on a workstation with an Apple M4 Pro, 24 GB unified memory, Python 3.9, NumPy 2.0, SciPy 1.13, scikit-learn

Table 7

Cross-dataset summary of the baseline comparison, averaged across the seven benchmark datasets ($n = 7$ per-dataset 5-seed means). Metrics are MCC, accuracy (ACC, %), macro-averaged true-positive rate (TPR, %), false-positive rate (FPR, %), precision (PPV, %), and F_1 (%). The p_t / p_w columns report paired t -test / Wilcoxon signed-rank p -values of CPAI-OvR vs. the baseline on the per-dataset MCC vector. The CPAI-OvA row is shown for reference. The full per-dataset breakdown is provided in Appendix E.

Model	MCC↑	ACC↑	TPR↑	FPR↓	PPV↑	F1↑	p_t vs. OvR	p_w vs. OvR
<i>Classical baselines</i>								
KNN	0.9158	98.24	92.36	1.038	93.04	92.35	0.056	0.016
LDA	0.7493	93.80	76.38	3.664	75.48	74.30	0.040	0.016
NuSVC	0.8692	97.14	87.89	1.690	89.80	88.33	0.009	0.016
SGD	0.8448	96.66	84.99	1.957	87.32	84.26	0.043	0.016
GauNB	0.7121	93.41	73.26	3.851	75.89	69.98	0.016	0.016
NC	0.5477	89.48	60.83	6.159	60.26	55.49	0.001	0.016
RNC	0.7789	95.14	78.08	2.843	87.73	79.67	0.004	0.016
KNFST	0.7790	95.12	80.37	2.897	79.84	79.00	0.073	0.016
<i>Modern baselines</i>								
LGBM	0.9718	99.45	97.48	0.325	97.55	97.49	0.680	0.469
MLP	0.9310	98.53	93.93	0.874	94.11	93.85	0.101	0.016
TabNet	0.9229	98.51	92.98	0.864	93.83	92.87	0.099	0.016
SAINT	0.9553	99.06	96.03	0.554	96.27	96.01	0.086	0.031
<i>Proposed</i>								
CPAI-OvA	0.9180	98.72	86.54	0.715	86.71	86.33	—	—
CPAI-OvR	0.9722	99.47	97.51	0.310	97.56	97.51	—	—

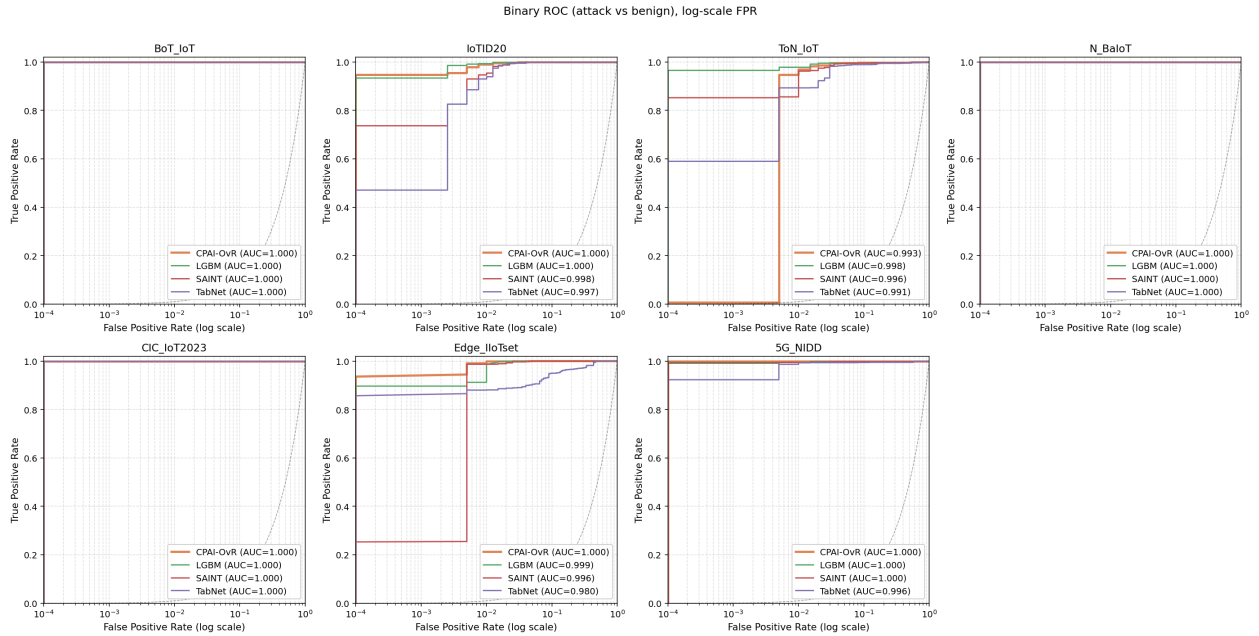


Figure 6: Binary attack-vs-benign ROC curves (logarithmic FPR axis) for CPAI-OvR and three modern baselines (LightGBM, TabNet, SAINT) on each of the seven benchmark datasets, seed 42. The log-scale FPR emphasizes the low-false-positive regime where IDS deployments operate. The four easier datasets (BoT-IoT, N-BaloT, CIC-IoT2023, 5G-NIDD) saturate at TPR ≈ 1 for all methods; on the three harder datasets (IoTID20, ToN-IoT, Edge-IIoTset) CPAI-OvR Pareto-dominates the deep-learning baselines for FPR $\leq 10^{-2}$.

1.6, PyTorch 2.8, LightGBM 4.6. Results are aggregated across the five seeds used in Section 5.5.

Training and inference runtime. Table 8 reports the mean fit and predict time of CPAI-OvA, CPAI-OvR, and the twelve baselines. CPAI-OvA and CPAI-OvR train in 1.0-4.7 seconds and 1.3-4.6 seconds respectively on every

dataset, and perform inference in under one second per 1,800-3,000-sample test batch. Compared with the strongest modern baselines, CPAI is approximately 2 $\times$ faster to train than LightGBM on Edge-IIoTset (4.58 s vs. 9.88 s), 5-10 $\times$ faster than TabNet, and 30-400 $\times$ faster than SAINT.

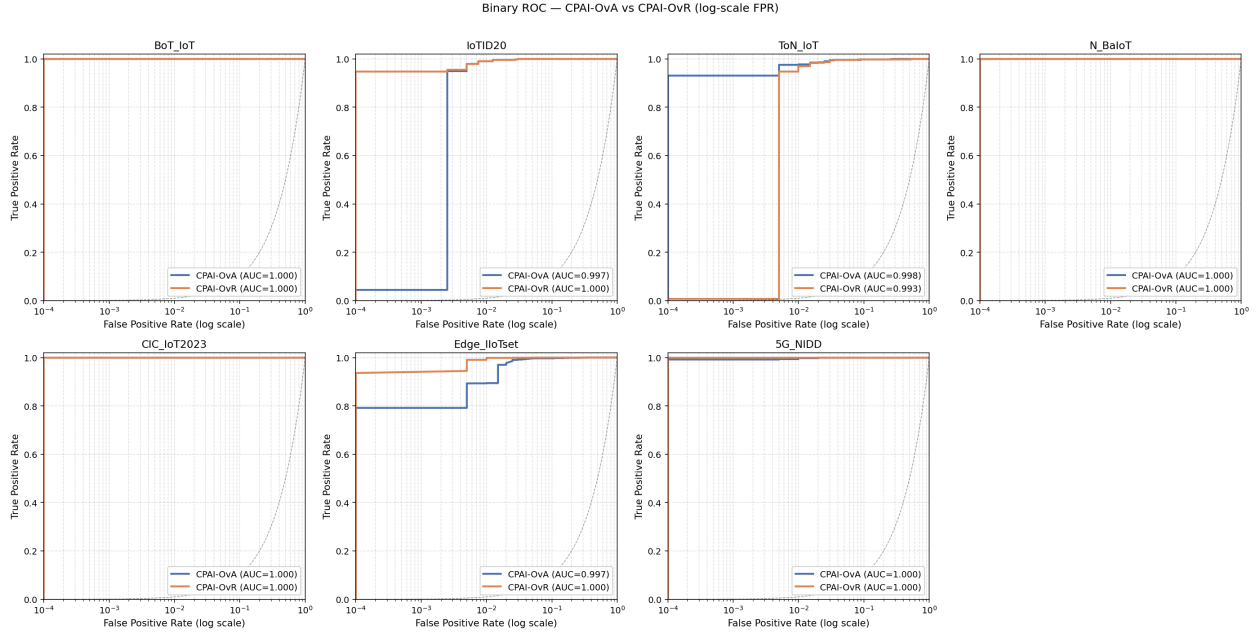


Figure 7: Head-to-head binary attack-vs-benign ROC curves for CPAI-OvA vs. CPAI-OvR on each of the seven datasets, log-scale FPR, seed 42. CPAI-OvR reaches the top-left corner at consistently lower FPR than CPAI-OvA. The gap is largest on Edge-IIoTset, where CPAI-OvR reaches TPR ≈ 0.9 at FPR = 10^{-2} while CPAI-OvA requires FPR $\approx 10^{-1}$ for the same recall.

OvA vs. OvR training-time reconciliation. The empirical parity between CPAI-OvA and CPAI-OvR in Table 8 warrants brief clarification relative to the $O(cTn^2)$ bound given in Section 3.5. That bound assumes c independent Conjugate Gradient (CG) solves, one per class, and is the relevant cost model for large-scale settings where direct factorization becomes infeasible. Our reference implementation instead performs a single Cholesky factorisation of $(K + \lambda I)$ at cost $O(n^3)$ and solves the c per-class OvR systems as back-substitutions against a stacked target matrix $B = [\beta^1, \dots, \beta^c] \in \mathbb{R}^{n \times c}$, for a marginal cost of $O(cn^2)$. Because $cn^2 \ll n^3$ at our operating sizes ($c \leq 15$, $n \lesssim 11,500$), and because BLAS-level multi-threading further amortises the c right-hand sides across CPU cores, OvA and OvR wall-clock training times stay within a $2\times$ factor on every benchmark, with OvR occasionally faster.

Memory footprint. Table 9 reports the peak resident set size of CPAI-OvA and CPAI-OvR at their best-per-dataset configuration. Peak memory ranges from 1.2 GB (5G-NIDD, BoT-IoT, N-BaIoT) to 2.8 GB (Edge-IIoTset, the largest training set at $n_{\text{train}} = 11,440$), with a mean of approximately 1.6 GB across the seven benchmarks. CPAI-OvA and CPAI-OvR sit within 25 MB of each other on every dataset because both variants share the same Gram-matrix construction and triangular-factorization path; the additional c right-hand sides used by CPAI-OvR add negligible memory.

Complexity. Training kernel CPAI requires one $n \times n$ kernel matrix build ($O(n^2d)$) and one LAPACK solve ($O(n^3)$), for an overall worst-case cost of $O(n^3)$. Inference costs $O(nd + nc)$ per test point. The asymptotic training

scaling matches KNFST but without the eigendecomposition, yielding a roughly $3\times$ constant-factor advantage in practice. Classical LDA trains in $O(nd^2 + d^3)$ and predicts in $O(d)$; faster per sample, but significantly less accurate on every benchmark dataset (mean MCC 0.7493 vs. 0.9722 for CPAI-OvR). For $n \gtrsim 50,000$ the $O(n^2)$ Gram-matrix memory becomes the binding constraint rather than the $O(n^3)$ arithmetic, and Nyström approximation or Random Fourier Features would be required to scale further (future work, Section 7).

IoT/edge deployability. At inference time, a trained CPAI model stores the training set $X \in \mathbb{R}^{n \times d}$ and the coefficient vector or matrix α . For our largest benchmark (Edge-IIoTset, $n_{\text{train}} = 11,440$, $d = 42$, $c = 15$), the total model footprint is $n(d + c) + d \approx 650,000$ numbers - approximately 5 MB in float64 and 2.6 MB in float32. A single prediction evaluates one kernel row ($O(nd)$) and c dot products ($O(nc)$); the measured inference latency is under 1 second per 1,800-3,000-sample test batch across all datasets, corresponding to sub-millisecond per-flow inference. Both the inference memory and latency envelopes fit the 1-4 GB RAM budget of typical IoT-gateway hardware (Raspberry Pi 4, NVIDIA Jetson Nano, commercial edge routers). In edge architectures, training is typically offloaded to a central controller, so the 1-5-second training cost is relevant only for periodic retraining, not per-packet inference.

6. Discussion

Building on the experimental results of Section 5, we summarise CPAI's practical advantages over the related

Table 8

Mean fit and inference wall-clock time (in seconds) across five seeds. For the proposed CPAI variants, the fit time includes one Cholesky factorisation of $(K + \lambda I)$ plus back-substitution against the (scalar for OvA, $n \times c$ matrix for OvR) target; see Section 5.6 for the OvA-vs-OvR runtime reconciliation.

Method	BoT-IoT	IoTID20	ToN-IoT	N-BalIoT	CIC-IoT2023	Edge-IIoTset	5G-NIDD
<i>Fit time (s) / Predict time (s)</i>							
KNN	0.00 / 0.021	0.00 / 0.031	0.00 / 0.022	0.00 / 0.039	0.00 / 0.027	0.00 / 0.055	0.00 / 0.026
LDA	0.00 / 0.001	0.02 / 0.000	0.01 / 0.000	0.03 / 0.001	0.01 / 0.000	0.01 / 0.001	0.01 / 0.000
NuSVC	0.60 / 0.168	1.12 / 0.336	0.64 / 0.312	1.15 / 0.529	0.90 / 0.335	2.00 / 1.055	0.66 / 0.271
SGD	0.02 / 0.000	0.22 / 0.000	0.11 / 0.000	0.21 / 0.000	0.06 / 0.000	0.48 / 0.000	0.05 / 0.000
GauNB	0.00 / 0.001	0.00 / 0.002	0.00 / 0.002	0.00 / 0.004	0.00 / 0.002	0.00 / 0.006	0.00 / 0.003
NC	0.00 / 0.002	0.00 / 0.002	0.00 / 0.002	0.00 / 0.004	0.00 / 0.002	0.00 / 0.004	0.00 / 0.003
RNC	0.00 / 0.018	0.00 / 0.032	0.00 / 0.032	0.00 / 0.042	0.00 / 0.034	0.00 / 0.072	0.00 / 0.028
KNFST	191.52 / 0.240	265.13 / 0.228	131.77 / 0.213	113.57 / 0.244	127.47 / 0.211	197.85 / 0.525	81.09 / 0.157
LGBM	2.68 / 0.013	3.21 / 0.015	3.67 / 0.019	5.00 / 0.021	2.98 / 0.013	9.88 / 0.064	4.38 / 0.018
MLP	0.31 / 0.001	1.51 / 0.001	1.48 / 0.001	1.15 / 0.001	0.54 / 0.001	2.68 / 0.002	0.92 / 0.001
TabNet	21.88 / 0.058	23.84 / 0.075	25.77 / 0.066	30.16 / 0.099	29.91 / 0.076	43.10 / 0.106	26.52 / 0.072
SAINT	130.25 / 0.500	454.84 / 1.351	206.94 / 0.672	596.91 / 1.682	247.02 / 0.784	285.82 / 0.904	201.49 / 0.635
CPAI-OvA	1.72 / 0.266	3.81 / 0.700	1.28 / 0.333	1.01 / 0.142	3.13 / 0.426	4.71 / 0.822	2.20 / 0.348
CPAI-OvR	1.54 / 0.296	2.11 / 0.700	1.33 / 0.342	2.45 / 0.903	1.61 / 0.442	4.58 / 0.891	1.34 / 0.346

Table 9

Peak resident set size (MB) of one full training run for CPAI-OvA and CPAI-OvR.

Dataset	n_{train}	Peak RSS
<i>CPAI-OvA</i>		
BoT-IoT	7,729	1,440
IoTID20	7,680	1,584
ToN-IoT	7,247	1,586
N-BalIoT	6,917	1,296
CIC-IoT2023	7,640	1,604
Edge-IIoTset	10,550	2,469
5G-NIDD	6,880	1,210
<i>CPAI-OvR</i>		
BoT-IoT	7,729	1,461
IoTID20	7,680	1,582
ToN-IoT	7,333	1,584
N-BalIoT	6,917	1,292
CIC-IoT2023	7,640	1,611
Edge-IIoTset	11,440	2,785
5G-NIDD	6,880	1,233
<i>Mean across datasets (both variants)</i>		1,624

approaches surveyed in Section 2, its structural limitations, and the data regimes in which it is expected to underperform.

6.1. Practical Advantages of CPAI

(i) *Numerical stability without the SSS bottleneck.* Unlike NFST, KNFST, and other null-space discriminant methods, CPAI does not rely on computing a zero-eigenvalue subspace. Training reduces to solving a single regularised linear system $(K + \lambda I)\alpha = \beta$ (Eq. 9), whose solution exists and is unique for any $\lambda > 0$, addressing the small-sample-size without any dimensionality-reduction heuristic.

(ii) *Uniformly competitive accuracy.* On the seven-benchmark comparison of Section 5.5, CPAI-OvR achieves the highest mean MCC and is either the best or within 0.005 MCC of the best method on every dataset - a cross-dataset consistency that almost no classical baseline or modern tabular model matches (Table 7, Appendix E).

(iii) *Deployability under IoT/edge constraints.* CPAI is a closed-form single-pass learner. The wall-clock training time, inference latency, and peak memory reported in Section 5.6 all fall within the 1-4 GB RAM envelope of typical IoT-gateway hardware (Raspberry Pi 4, Jetson Nano, commercial edge routers), unlike deep tabular baselines such as SAINT and TabNet, whose training costs may be higher.

(iv) *Robustness to regularisation.* The sensitivity analysis in Appendix C shows that CPAI-OvR is robust across four orders of magnitude of the ridge parameter ($\lambda \in [10^{-9}, 10^{-3}]$; MCC variation below 0.005 on every dataset).

6.2. Limitations

Two limitations merit explicit discussion. First, the $O(n^2)$ memory and $O(n^3)$ training-time scaling of kernel CPAI become bottlenecks for large-scale deployments. For training-set sizes beyond approximately 50,000 samples, we expect direct kernel construction to become infeasible, and

Nyström approximation or Random Fourier Features would be required to scale further; we leave this investigation for future work. Second, CPAI's performance is sensitive to the choice of training strategy and kernel: the linear and one-dimensional OvA variants underperform on non-positive-definite kernels (sigmoid, Sobolev; see Section 5.4), so the kernelized CPAI-OvR configuration is the recommended default.

6.3. When CPAI May Fail

Beyond the structural limitations above, our experiments identify two data-driven regimes in which CPAI is expected to underperform. First, CPAI lacks an intrinsic label-robustification mechanism, so severe label noise or mislabelled attack samples can mis-anchor the class plateau and degrade separability. Second, extreme class imbalance leaves the target plateau insufficiently anchored, resulting in a brittle discriminant direction. For kernel selection in novel domains, we recommend starting with the Laplacian kernel (best mean MCC across all seven datasets).

7. Conclusion and Future work

This paper introduced CPAI, a novel discriminant analysis framework specifically engineered for anomaly detection in resource-constrained IoT environments. By explicitly controlling projection targets and leveraging the kernel trick, CPAI fundamentally resolves the long-standing singularity problem inherent in prior null-space methods. Our theoretical analysis, complemented by spectral dynamics, establishes that CPAI provides a bounded, geometrically stable projection space, ensuring high-confidence consensus over legitimate traffic manifolds. The extensive empirical evaluation across seven large-scale IoT benchmarks demonstrates CPAI's superior performance, consistently achieving higher accuracy and stability compared to existing distance-based algorithms. The findings emphasized that CPAI bridges the gap between the efficiency of shallow methods and the separation power of supervised discriminant analysis, making it uniquely suited for resource-constrained IoT gateways. A full discussion of CPAI's practical advantages, structural limitations, and the data regimes in which it is expected to underperform is provided in Section 6.

Looking ahead, the present work opens several promising future directions:

- *Scalable kernel approximations.* Combining CPAI with Nyström approximation or Random Fourier Features would remove the $n \gtrsim 50,000$ bottleneck and make CPAI usable on massive-scale IoT traffic streams.
- *Interpretable intrusion detection.* In security operations, knowing *why* a flow was flagged is as important as detection accuracy. Inverting the CPAI discriminant directions to identify which network features cause the structural drop would give analysts feature-level explanations for every alert.

- *Online and incremental CPAI.* IoT traffic changes over time. An incremental version of CPAI that updates the target plateau without full retraining would improve robustness against concept drift and evolving zero-day attacks.
- *Cross-domain extensions.* The idea of target-controlled manifold collapse is not specific to IoT. Natural next applications include industrial sensor fault diagnosis, medical image anomaly detection, and fraud detection in financial transactions.

Acknowledgment

This research was supported by the VNUHCM-University of Information Technology's Scientific Research Support Fund.

References

- [1] Ahmad, R., Alsmadi, I., Alhamdani, W., Tawalbeh, L., 2022. A comprehensive deep learning benchmark for iot ids. *Computers & Security* 114, 102588.
- [2] Aldhaferi, A., Alwahedi, F., Ferrag, M.A., Battah, A., 2024. Deep learning for cyber threat detection in iot networks: A review. *Internet of Things and cyber-physical systems* 4, 110–128.
- [3] Ali, T., Chaudhuri, S., 2023. Theoretical analysis of null foley-sammon transform and its implications. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 45, 6445–6459. doi:10.1109/TPAMI.2022.3213069.
- [4] Alsaedi, A., Moustafa, N., Tari, Z., Mahmood, A., Anwar, A., 2020. Ton_iot telemetry dataset: A new generation dataset of iot and iiot for data-driven intrusion detection systems. *Ieee Access* 8, 165130–165150. doi:10.1109/ACCESS.2020.3022862.
- [5] Arik, S.Ö., Pfister, T., 2021. Tabnet: Attentive interpretable tabular learning, in: *Proceedings of the AAAI conference on artificial intelligence*, pp. 6679–6687.
- [6] Azam, Z., Islam, M.M., Huda, M.N., 2023. Comparative analysis of intrusion detection systems and machine learning based model analysis through decision tree. *IEEE Access* 11, 64516–64535. doi:10.1109/ACCESS.2023.3289053.
- [7] Bentley, J.L., 1975. Multidimensional binary search trees used for associative searching. *Communications of the ACM* 18, 509–517. doi:10.1145/361002.361007.
- [8] Böcekçi, S.C., Yıldız, K., 2026. Adversarial attack detection in resource-constrained environments: A stable and sequential federated learning architecture with tinyllama-1.1 b. Preprint. Research Square. Doi: 10.21203/rs.3.rs-8613070/v1.
- [9] Bodesheim, P., Freytag, A., Rodner, E., Kemmler, M., Denzler, J., 2013. Kernel null space methods for novelty detection, in: *2013 IEEE Conference on Computer Vision and Pattern Recognition*, pp. 3374–3381. doi:10.1109/CVPR.2013.433.
- [10] Breunig, M.M., Kriegel, H.P., Ng, R.T., Sander, J., 2000. Lof: identifying density-based local outliers, in: *Proceedings of the 2000 ACM SIGMOD international conference on Management of data*, pp. 93–104.
- [11] Buyuktanir, B., Altinkaya, Ş., Karatas Baydogmus, G., Yildiz, K., 2025. Federated learning in intrusion detection: advancements, applications, and future directions. *Cluster Computing* 28, 473.
- [12] Cerruto, F., Cirillo, S., Desiato, D., Gambardella, S.M., Polese, G., 2022. Social network data analysis to highlight privacy threats in sharing data. *Journal of Big Data* 9, 19.
- [13] Chicco, D., Jurman, G., 2020. The advantages of the matthews correlation coefficient (mcc) over f1 score and accuracy in binary classification evaluation. *BMC Genomics* 21. doi:10.1186/s12864-019-6413-7.

- [14] Cil, A.E., Yildiz, K., Buldu, A., 2021. Detection of ddos attacks with feed forward based deep neural network model. *Expert Systems with Applications* 169, 114520.
- [15] Cirillo, S., Desiato, D., Scalera, M., Solimando, G., 2023. A visual privacy tool to help users in preserving social network data, in: *Joint Proceedings of the Workshops, Work in Progress Demos and Doctoral Consortium at IS-EUD 2023*, Cagliari, Italy.
- [16] Falcão, F., Zoppi, T., Silva, C.B.V., Santos, A., Fonseca, B., Ceccarelli, A., Bondavalli, A., 2019. Quantitative comparison of unsupervised anomaly detection algorithms for intrusion detection, in: *Proceedings of the 34th ACM/SIGAPP symposium on applied computing*, pp. 318–327.
- [17] Fawcett, T., 2006. An introduction to roc analysis. *Pattern Recognition Letters* 27, 861–874. doi:10.1016/j.patrec.2005.10.010.
- [18] Ferrag, M.A., Friha, O., Hamouda, D., Maglaras, L., Janicke, H., 2022. Edge-iiotset: A new comprehensive realistic cyber security dataset of iot and iiot applications for centralized and federated learning. *IEEE Access* 10, 40281–40306.
- [19] Fisher, R.A., 1936. The use of multiple measurements in taxonomic problems. *Annals of Eugenics* 7, 179–188. doi:https://doi.org/10.1111/j.1469-1809.1936.tb02137.x.
- [20] Foley, D.H., Sammon, J.W., 1975. An optimal set of discriminant vectors. *IEEE Transactions on Computers* C-24, 281–289. doi:10.1109/T-C.1975.224208.
- [21] Gautam, C., Tiwari, A., Tanveer, M., 2019. Koc+: Kernel ridge regression based one-class classification using privileged information. *Information Sciences* 504, 324–333. URL: https://www.sciencedirect.com/science/article/pii/S0020025519306589, doi:https://doi.org/10.1016/j.ins.2019.07.052.
- [22] Golub, G.H., Van Loan, C.F., 2013. *Matrix Computations*. Fourth ed., JHU Press. URL: http://www.cs.cornell.edu/cv/GVL4/golubandvanloan.htm.
- [23] Guo, Y.F., Wu, L., Lu, H., Feng, Z., Xue, X., 2006. Null foley-sammon transform. *Pattern Recognition* 39, 2248–2251. doi:https://doi.org/10.1016/j.patcog.2006.05.009.
- [24] Hand, D.J., Till, R.J., 2001. A simple generalisation of the area under the roc curve for multiple class classification problems. *Machine Learning* 45, 171–186. doi:10.1023/A:1010920819831.
- [25] Heidari, A., Jabraeli Jamali, M.A., 2023. Internet of things intrusion detection systems: a comprehensive review and future directions. *Cluster Computing* 26, 3753–3780.
- [26] Jamalipour, A., Murali, S., 2021. A taxonomy of machine-learning-based intrusion detection systems for the internet of things: A survey. *IEEE Internet of Things Journal* 9, 9444–9466.
- [27] John, G.H., Langley, P., 1995. Estimating continuous distributions in Bayesian classifiers, in: *Proceedings of the Eleventh Conference on Uncertainty in Artificial Intelligence (UAI)*, Morgan Kaufmann, San Francisco, CA, USA, pp. 338–345.
- [28] Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., Liu, T.Y., 2017. Lightgbm: A highly efficient gradient boosting decision tree. *Advances in neural information processing systems* 30.
- [29] Koroniotis, N., Moustafa, N., Sitnikova, E., Turnbull, B., 2019. Towards the development of realistic botnet dataset in the internet of things for network forensic analytics: Bot-iiot dataset. *Future Generation Computer Systems* 100, 779–796.
- [30] Leng, Q., Qi, H., Miao, J., Zhu, W., Su, G., 2015. One-class classification with extreme learning machine. *Mathematical Problems in Engineering* 2015, 412957. URL: https://onlinelibrary.wiley.com/doi/abs/10.1155/2015/412957, doi:https://doi.org/10.1155/2015/412957, arXiv:https://onlinelibrary.wiley.com/doi/pdf/10.1155/2015/412957.
- [31] Lepilov, M., 2024. Fast spectrum estimation of some kernel matrices. URL: https://arxiv.org/abs/2411.00657, arXiv:2411.00657.
- [32] Lin, Y., Gu, G., Liu, H., Shen, J., 2008. Kernel null foley-sammon transform, in: *2008 International Conference on Computer Science and Software Engineering*, pp. 981–984. doi:10.1109/CSSE.2008.621.
- [33] Matthews, B.W., 1975. Comparison of the predicted and observed secondary structure of t4 phage lysozyme. *Biochimica et Biophysica Acta (BBA) - Protein Structure* 405, 442–451. doi:10.1016/0005-2795(75)90109-9.
- [34] Meidan, Y., Bohadana, M., Mathov, Y., Mirsky, Y., Shabtai, A., Breitenbacher, D., Elovici, Y., 2018. N-baiot—network-based detection of iot botnet attacks using deep autoencoders. *IEEE Pervasive Computing* 17, 12–22.
- [35] Mercer, J., 1909. Functions of positive and negative type, and their connection with the theory of integral equations. *Philosophical Transactions of the Royal Society A* 209, 415–446.
- [36] Mohanta, B.K., Jena, D., Satapathy, U., Patnaik, S., 2020. Survey on iot security: Challenges and solution using machine learning, artificial intelligence and blockchain technology. *Internet of Things* 11, 100227.
- [37] Neto, E.C.P., Dadkhah, S., Ferreira, R., Zohourian, A., Lu, R., Ghorbani, A.A., 2023. Ciciot2023: A real-time dataset and benchmark for large-scale attacks in iot environment. *Sensors* 23, 5941.
- [38] Nguyen, X.H., Nguyen, X.D., Huynh, H.H., Le, K.H., 2022. Re-alguard: A lightweight network intrusion detection system for iot gateways. *Sensors* 22, 432.
- [39] Peterson, L.E., 2009. K-nearest neighbor. *Scholarpedia* 4, 1883.
- [40] van Rijsbergen, C.J., 1979. *Information Retrieval*. 2nd ed., Butterworth-Heinemann.
- [41] Ruff, L., Vandermeulen, R., Goernitz, N., Deecke, L., Siddiqui, S.A., Binder, A., Müller, E., Kloft, M., 2018. Deep one-class classification, in: *International conference on machine learning*, PMLR. pp. 4393–4402.
- [42] Samarakoon, S., Siriwardhana, Y., Porambage, P., Liyanage, M., Chang, S.Y., Kim, J., Kim, J., Ylianttila, M., 2022. 5g-nidd: A comprehensive network intrusion detection dataset generated over 5g wireless network. *arXiv preprint arXiv:2212.01298*.
- [43] Saunders, C., Gammernan, A., Vovk, V., 1998. Ridge regression learning algorithm in dual variables, in: *Proceedings of the Fifteenth International Conference on Machine Learning*, Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, p. 515–521.
- [44] Schölkopf, B., Herbrich, R., Smola, A.J., 2001a. A generalized representer theorem, in: *Computational Learning Theory*, Springer Berlin Heidelberg, pp. 416–426. doi:10.1007/3-540-44581-4_27.
- [45] Schölkopf, B., Platt, J.C., Shawe-Taylor, J., Smola, A.J., Williamson, R.C., 2001b. Estimating the support of a high-dimensional distribution. *Neural computation* 13, 1443–1471.
- [46] Schölkopf, B., Smola, A., Müller, K.R., 1998. Nonlinear component analysis as a kernel eigenvalue problem. *Neural Computation* 10, 1299–1319. doi:10.1162/089976698300017467.
- [47] Schölkopf, B., Smola, A.J., Williamson, R.C., Bartlett, P.L., 2000. New support vector algorithms. *Neural Computation* 12, 1207–1245. doi:10.1162/089976600300015565.
- [48] Shewchuk, J.R., 1994. *An Introduction to the Conjugate Gradient Method Without the Agonizing Pain*. Technical Report. Carnegie Mellon University. Pittsburgh, PA, USA. Edition 1.25.
- [49] Sokolova, M., Lapalme, G., 2009. A systematic analysis of performance measures for classification tasks. *Information Processing & Management* 45, 427–437. doi:10.1016/j.ipm.2009.03.002.
- [50] Somepalli, G., Goldblum, M., Schwarzschild, A., Bruss, C.B., Goldstein, T., 2021. Saint: Improved neural networks for tabular data via row attention and contrastive pre-training. *arXiv preprint arXiv:2106.01342*.
- [51] Tax, D.M.J., Duin, R.P.W., 2004. Support vector data description. *Mach. Learn.* 54, 45–66. URL: https://doi.org/10.1023/B:MACH.0000008084.60811.49, doi:10.1023/B:MACH.0000008084.60811.49.
- [52] Tibshirani, R., Hastie, T., Narasimhan, B., Chu, G., 2002. Diagnosis of multiple cancer types by shrunken centroids of gene expression. *Proceedings of the National Academy of Sciences* 99, 6567–6572. doi:10.1073/pnas.082099299.
- [53] Ullah, I., Mahmoud, Q.H., 2020. A scheme for generating a dataset for anomalous activity detection in iot networks, in: *Canadian conference on artificial intelligence*, Springer. pp. 508–520.

- [54] Wathen, A.J., 2015. Preconditioning. Acta Numerica 24, 329–376. doi:10.1017/S0962492915000021.
- [55] Zhang, T., 2004. Solving large scale linear prediction problems using stochastic gradient descent algorithms, in: Proceedings of the Twenty-First International Conference on Machine Learning (ICML), ACM, Banff, Alberta, Canada. pp. 919–926. doi:10.1145/1015330.1015332.

A. Overview of NFST and KNFST

In this appendix, we provide a brief overview of NFST and its kernelized version (KNFST) to make the paper self-contained and to provide context for the proposed CPAI method. Assume the dataset consists of n samples (denoted by x^i) belonging to c classes $\{x_1^i\}_{i=1}^{n_1}, \dots, \{x_c^i\}_{i=1}^{n_c}$ where n_j and $x_j^i \in \mathbb{R}^d$ denote the sample size and the i^{th} instance of the j^{th} class, respectively. Let us define $\bar{x}_j = \frac{1}{n_j} \sum_{i=1}^{n_j} x_j^i$ and $\bar{x} = \frac{1}{n} \sum_{i=1}^n x^i$ as the mean of the j^{th} class and the total mean of the data, respectively. Foley-Sammon Transform (FST) [20] aims to find an optimal discriminant vector $\varphi \in \mathbb{R}^d$ that maximizes the Fisher criterion [19], under orthonormal constraints:

$$J(\varphi) = \frac{\varphi^T S_b \varphi}{\varphi^T S_w \varphi}, \quad (27)$$

where $S_w \in \mathbb{R}^{d \times d}$ and $S_b \in \mathbb{R}^{d \times d}$ are the within-class scatter matrix and between-class scatter matrices, respectively, defined as

$$S_w := \sum_{j=1}^c \sum_{i=1}^{n_j} (x_j^i - \bar{x}_j)(x_j^i - \bar{x}_j)^T$$

$$S_b := \sum_{j=1}^c n_j (\bar{x}_j - \bar{x})(\bar{x}_j - \bar{x})^T.$$

If S_w is nonsingular, φ_1 is the unit eigenvector corresponding to the maximum eigenvalue of $S_w^{-1} S_b$. The remaining discriminant directions are obtained by solving

$$\max_{\varphi_i} J(\varphi) \quad \text{s.t., } \varphi_i^T \varphi_j = \delta_{ij}, \quad j = 1, \dots, i-1. \quad (28)$$

In contrast to FST, Null Foley-Sammon Transform (NFST) [23] aims to find φ - a so-called null projecting direction (NPD) - which satisfies:

$$\varphi^T S_w \varphi = 0, \quad (29)$$

$$\varphi^T S_b \varphi > 0. \quad (30)$$

Then the data have the best separability on the direction φ according to the Fisher discriminant criterion. For the **small-sample-size (SSS) case** (i.e., $n \leq d$), we additionally assume that the sample vectors are **linearly** independent. Let us define $Z_t = \{x \in \mathbb{R}^d \mid S_t x = 0\}$ where $S_t = S_w + S_b$ is the total scatter matrix, and $Z_t^\perp$ denotes the orthogonal complement space of Z_t . It can be shown that

Theorem 5. (see [23]) *If the sample vectors are linearly independent, there are $c - 1$ NPDs $\varphi_i \in (Z_w \cap Z_t^\perp)$ where $Z_w = \{x \in \mathbb{R}^d \mid S_w x = 0\}$.*

To make it efficient for $d > n$ case, the nonlinear version of the theory was proposed; see the details in [32]. The key point is to take each column vector of K as a sample vector. Here, K means to be generated by the dot product of all the training samples in the implicit feature space, which is similar to (8). The procedure of computing KNFST can be summarized as follows [32]:

- Step 1. Compute the kernel matrix K using (8).
- Step 2. Compute the orthogonal basis $(\tau_1, \dots, \tau_{n-c})$ from the set of **centered** vectors $\{K_{\cdot 1} - \bar{k}, K_{\cdot 2} - \bar{k}, \dots, K_{\cdot n} - \bar{k}\}$ where $\bar{k} = \frac{1}{n} \sum_{i=1}^n K_{\cdot i}$.
- Step 3. Set $P = (\tau_1, \dots, \tau_{n-c})$ and compute $S = P^T K (I - L) P$ where I is the $n \times n$ identity matrix, and $L = (L_l)_{l=1, \dots, c}$ is a $n \times n$ block diagonal matrix with entries $L_{ii} = 1/n_j$ if sample i belongs to class j . Solving $S\eta = 0$ yields vectors $\eta_1, \dots, \eta_{c-1}$.
- Step 4. KNFST is then given by AQ , where $A = (\eta_1, \dots, \eta_{c-1})$.

Ali and Chaudhuri [3] showed that the singularities of different classes are not necessarily distinct, losing their class identities, which is a major limitation of NFST-based classifiers.

B. Comparison between CPAI and NFST

Recent findings in [3] highlight a critical vulnerability in NFST-based classifiers, where singularities of different classes may overlap. This phenomenon leads to a loss of class distinctiveness, thereby compromising categorical integrity and fundamentally limiting the effectiveness of prior null-space discriminant frameworks.

1. *Addressing the singularity problem:* NFST relies on the constraints $\varphi^T S_w \varphi = 0$ and $\varphi^T S_b \varphi > 0$. However, it offers no explicit control over the final locations in the projection space. CPAI directly addresses this by allowing the user to predefine distinct target projection values (b_j) for each class. The algorithm then explicitly constructs discriminant vectors to map samples towards these separated coordinates by solving $(K + \lambda I)\alpha = \beta$.
2. *Handling data dimensionality:* Linear NFST fundamentally requires linear independence of sample vectors, which generally necessitates $n \leq d$. Kernel CPAI overcomes this limitation by working with the $n \times n$ kernel matrix K . As long as $\lambda > 0$, a unique solution α exists regardless of the dimension's assumption.
3. *Implementation and Kernel choice:* The effectiveness of Kernel CPAI relies on selecting a suitable kernel function κ that results in a well-conditioned (or regularized) and invertible kernel matrix K , and captures the underlying data structure. Our experiments demonstrate that appropriate kernel choices (like Laplacian, RBF, Abel in our tests) significantly enhance performance over the linear case and potentially problematic kernels like Sigmoid. This highlights the importance of kernel selection, a common aspect of kernel methods.

Table 10

MCC of CPAI-OvA as a function of λ (best configuration per dataset; seed = 42).

Dataset	10^{-9}	10^{-7}	10^{-5}	10^{-3}	10^{-1}
BoT-IoT	0.9994	0.9994	0.9994	0.9994	0.9994
IoTID20	0.9154	0.9154	0.9135	0.8908	0.7974
ToN-IoT	0.9581	0.9581	0.9581	0.9593	0.9341
N-BaIoT	0.9676	0.9676	0.9676	0.9197	0.8818
CIC-IoT2023	0.9957	0.9957	0.9957	0.9957	0.9936
Edge-IIoTset	0.6247	0.6247	0.6237	0.6109	0.4867
5G-NIDD	0.9664	0.9664	0.9664	0.9664	0.9577

Table 11

MCC of CPAI-OvR as a function of λ (best configuration per dataset; seed = 42).

Dataset	10^{-9}	10^{-7}	10^{-5}	10^{-3}	10^{-1}
BoT-IoT	1.0000	1.0000	1.0000	1.0000	1.0000
IoTID20	0.9469	0.9469	0.9475	0.9376	0.8972
ToN-IoT	0.9921	0.9921	0.9921	0.9909	0.9890
N-BaIoT	0.9938	0.9938	0.9938	0.9470	0.9445
CIC-IoT2023	0.9993	0.9993	0.9993	0.9993	0.9986
Edge-IIoTset	0.8835	0.8835	0.8835	0.8907	0.8858
5G-NIDD	0.9963	0.9963	0.9963	0.9969	0.9931

4. **Spectral Behavior:** Unlike NFST’s rigid binary selection of directions, CPAI performs continuous spectral weighting. By utilizing the scalar S_j and the Tikhonov regularization term (λ), CPAI ensures that global invariant features are preserved while singular amplification of noise is suppressed.

C. Sensitivity Analysis: Regularization Parameter λ

To support our default choices of $\lambda = 10^{-9}$ for CPAI-OvA and $\lambda = 10^{-5}$ for CPAI-OvR, we report the sensitivity of both variants to λ across all seven benchmark datasets. For each dataset the best (kernel, scaler, poly) configuration from the main comparison is held fixed, and only λ is varied across $\{10^{-9}, 10^{-7}, 10^{-5}, 10^{-3}, 10^{-1}\}$ (single seed, seed = 42, per configuration).

CPAI-OvR is essentially flat across $\lambda \in [10^{-9}, 10^{-3}]$ on every dataset (MCC variation below 0.005). CPAI-OvA is similarly flat across $\lambda \in [10^{-9}, 10^{-5}]$, with mild degradation (0.05–0.15 MCC drop) appearing at larger λ on three datasets with larger or harder-conditioned kernel matrices. Both default values therefore lie safely within the flat region on all seven datasets.

D. Confusion Matrices for CPAI-OvA and CPAI-OvR Across the Seven Benchmark Datasets

Figure 8 shows the per-class confusion matrices of both CPAI variants on all seven benchmark datasets, arranged as a 7×2 grid (one row per dataset; CPAI-OvA on the left,

CPAI-OvR on the right). Within every panel, rows are true classes, columns are predicted classes, and the benign class is labeled *Normal*.

The CPAI-OvA panels carry an extra *<other>* row and column that records the OvA reject code: a prediction of -1 , issued when the single scalar discriminant of CPAI-OvA falls outside the training label range $[\min y, \max y]$. The reject code has no analog on the CPAI-OvR side, where the $\arg \min_j |f_j(x) - 1|$ rule always selects one of the c trained classes. We retain this slot because it provides a natural entry point for extending CPAI to multiclass novelty detection: a “-1” prediction is by construction a sample that does not project cleanly onto any trained class target, and can therefore be flagged as a candidate novel attack.

Across the grid, both variants exhibit strong diagonal dominance, with CPAI-OvR producing visibly cleaner diagonals than CPAI-OvA, consistent with the aggregate metrics in Section 5.4. Residual off-diagonal mass concentrates on semantically adjacent attack families rather than leaking into the *Normal* row, so the benign class is classified with near-perfect precision on every dataset and every variant.

E. Per-Dataset Baseline Comparison Results

The summary of Section 5.5 (Table 7) is condensed across the seven benchmark datasets; this appendix gives the per-dataset breakdown. For each benchmark, we report the six classification metrics of Section 5.2 as mean $\pm$ std over five seeds, followed by two paired-test p -values against each CPAI variant: p_t is a two-sided paired t -test and p_w is a Wilcoxon signed-rank test, both computed on the 5-seed MCC vector. The CPAI rows carry the chosen (kernel, scaler, poly) configuration. An entry of $p \approx 0.062$ is the lowest achievable Wilcoxon value with $n = 5$ paired observations, and indicates that the CPAI variant strictly dominated the baseline at every seed. Table numbers in this appendix (Tables 12 through 18) follow the dataset order of Section 5.1.

CRedit authorship contribution statement

Van-Hoi Nguyen: Conceptualization, Methodology, Validation, Formal analysis, Writing - Original Draft. **Xuan-Ha Nguyen:** Methodology, Software, Data curation, Investigation, Writing - Original Draft. **Kim-Hung Le:** Conceptualization, Methodology, Project administration, Writing - Review & Editing.

Confusion matrices — best per-dataset run for CPAI-OvA (left) and CPAI-OvR (right)

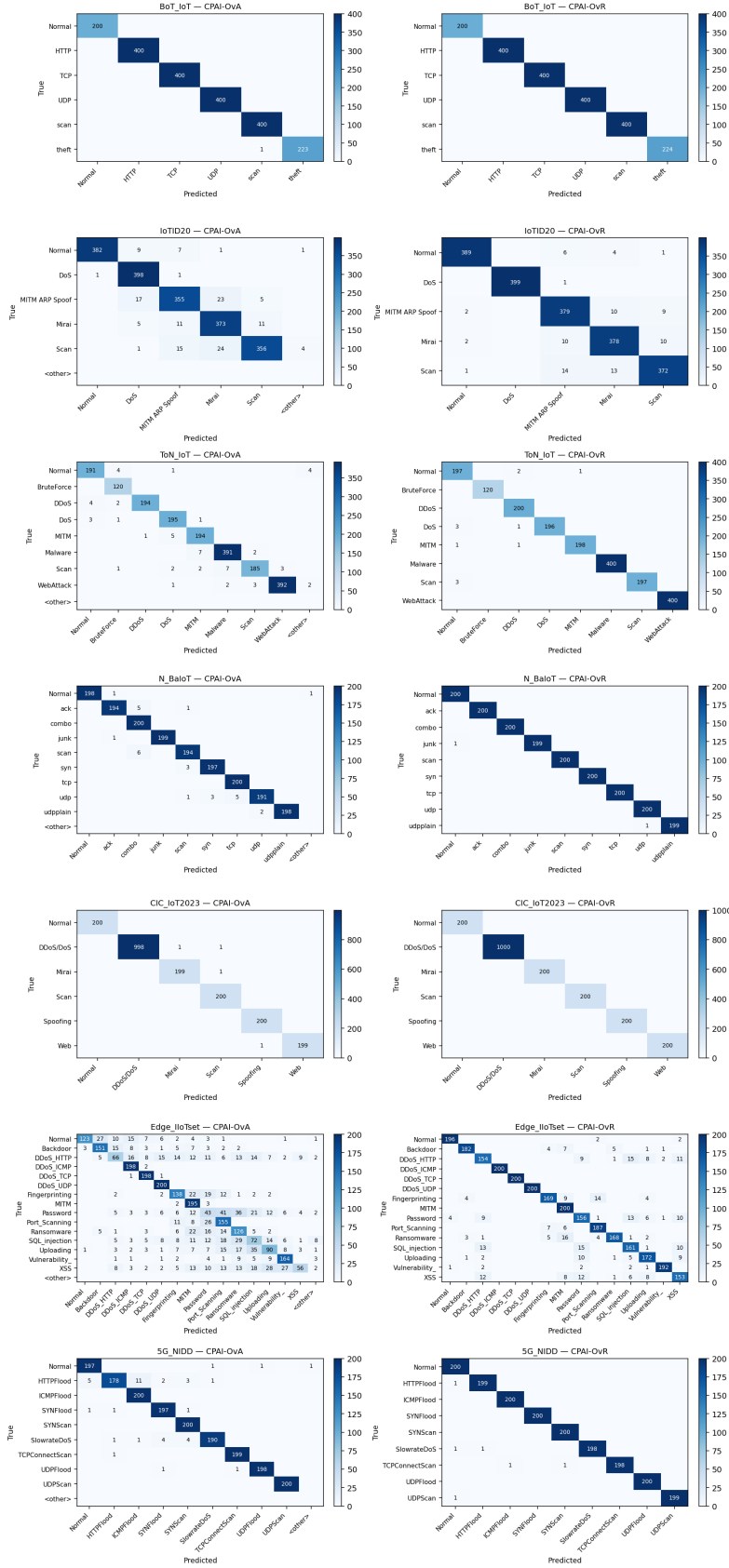


Figure 8: Confusion matrices of CPAI-OvA (left column) and CPAI-OvR (right column) across the seven benchmark datasets. Rows are true classes; columns are predicted classes.

Table 12

Per-dataset baseline comparison on BoT-IoT. Mean $\pm$ std over five seeds. Paired t -test (p_t) and Wilcoxon signed-rank (p_w) p -values are computed on the 5-seed MCC vector against CPAI-OvA and CPAI-OvR.

Model	MCC $\uparrow$	ACC $\uparrow$	TPR $\uparrow$	FPR $\downarrow$	PPV $\uparrow$	F1 $\uparrow$	p_t OvA	p_w OvA	p_t OvR	p_w OvR
KNN	0.9927 $\pm$ 0.0014	99.80 $\pm$ 0.04	99.47 $\pm$ 0.12	0.121 $\pm$ 0.024	99.44 $\pm$ 0.13	99.45 $\pm$ 0.12	0.002	0.062	0.000	0.062
LDA	0.9959 $\pm$ 0.0015	99.89 $\pm$ 0.04	99.72 $\pm$ 0.10	0.067 $\pm$ 0.025	99.72 $\pm$ 0.10	99.72 $\pm$ 0.10	0.051	0.062	0.004	0.062
NuSVC	0.9448 $\pm$ 0.0044	98.47 $\pm$ 0.12	95.89 $\pm$ 0.37	0.917 $\pm$ 0.073	96.29 $\pm$ 0.31	95.99 $\pm$ 0.34	0.000	0.062	0.000	0.062
SGD	0.9920 $\pm$ 0.0023	99.78 $\pm$ 0.06	99.33 $\pm$ 0.26	0.132 $\pm$ 0.037	99.45 $\pm$ 0.15	99.39 $\pm$ 0.20	0.012	0.062	0.001	0.062
GauNB	0.9812 $\pm$ 0.0060	99.48 $\pm$ 0.16	98.57 $\pm$ 0.47	0.310 $\pm$ 0.099	98.71 $\pm$ 0.41	98.63 $\pm$ 0.44	0.004	0.062	0.002	0.062
NC	0.8960 $\pm$ 0.0038	97.09 $\pm$ 0.11	90.14 $\pm$ 0.71	1.747 $\pm$ 0.068	93.52 $\pm$ 0.23	91.22 $\pm$ 0.55	0.000	0.062	0.000	0.062
RNC	0.9480 $\pm$ 0.0079	98.54 $\pm$ 0.23	94.38 $\pm$ 0.84	0.875 $\pm$ 0.136	96.99 $\pm$ 0.38	95.34 $\pm$ 0.70	0.000	0.062	0.000	0.062
KNFST	0.9911 $\pm$ 0.0035	99.76 $\pm$ 0.10	99.20 $\pm$ 0.36	0.146 $\pm$ 0.057	99.36 $\pm$ 0.27	99.27 $\pm$ 0.32	0.022	0.062	0.005	0.062
LGBM	0.9988 $\pm$ 0.0013	99.97 $\pm$ 0.03	99.92 $\pm$ 0.09	0.020 $\pm$ 0.021	99.92 $\pm$ 0.09	99.92 $\pm$ 0.09	0.804	0.812	0.103	0.109
MLP	0.9958 $\pm$ 0.0029	99.88 $\pm$ 0.08	99.68 $\pm$ 0.22	0.069 $\pm$ 0.048	99.65 $\pm$ 0.21	99.67 $\pm$ 0.21	0.110	0.125	0.032	0.062
TabNet	0.9990 $\pm$ 0.0009	99.97 $\pm$ 0.02	99.93 $\pm$ 0.06	0.016 $\pm$ 0.015	99.90 $\pm$ 0.10	99.91 $\pm$ 0.08	0.566	0.438	0.078	0.068
SAINT	0.9973 $\pm$ 0.0058	99.92 $\pm$ 0.16	99.80 $\pm$ 0.43	0.045 $\pm$ 0.096	99.68 $\pm$ 0.69	99.73 $\pm$ 0.58	0.673	0.625	0.350	0.180
CPAI-OvA	0.9986 $\pm$ 0.0013	99.96 $\pm$ 0.04	99.86 $\pm$ 0.09	0.024 $\pm$ 0.022	99.89 $\pm$ 0.10	99.88 $\pm$ 0.09	—	—	—	—
CPAI-OvR	1.0000 $\pm$ 0.0000	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00	0.000 $\pm$ 0.000	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00	—	—	—	—

Table 13

Per-dataset baseline comparison on IoTID20.

Model	MCC $\uparrow$	ACC $\uparrow$	TPR $\uparrow$	FPR $\downarrow$	PPV $\uparrow$	F1 $\uparrow$	p_t OvA	p_w OvA	p_t OvR	p_w OvR
KNN	0.8094 $\pm$ 0.0106	93.85 $\pm$ 0.34	84.63 $\pm$ 0.84	3.842 $\pm$ 0.211	85.32 $\pm$ 0.84	84.68 $\pm$ 0.83	0.000	0.062	0.000	0.062
LDA	0.7176 $\pm$ 0.0102	90.86 $\pm$ 0.32	77.16 $\pm$ 0.81	5.710 $\pm$ 0.202	79.60 $\pm$ 0.82	77.71 $\pm$ 0.74	0.000	0.062	0.000	0.062
NuSVC	0.7363 $\pm$ 0.0217	91.52 $\pm$ 0.69	78.81 $\pm$ 1.73	5.298 $\pm$ 0.433	79.55 $\pm$ 1.85	78.95 $\pm$ 1.76	0.000	0.062	0.000	0.062
SGD	0.6828 $\pm$ 0.0668	89.19 $\pm$ 2.80	72.97 $\pm$ 6.99	6.758 $\pm$ 1.748	78.17 $\pm$ 4.83	71.69 $\pm$ 9.55	0.002	0.062	0.001	0.062
GauNB	0.5352 $\pm$ 0.0056	82.80 $\pm$ 0.69	56.99 $\pm$ 1.72	10.753 $\pm$ 0.431	75.19 $\pm$ 2.04	54.89 $\pm$ 3.28	0.000	0.062	0.000	0.062
NC	0.4145 $\pm$ 0.0129	80.94 $\pm$ 0.40	52.34 $\pm$ 1.00	11.915 $\pm$ 0.250	49.22 $\pm$ 1.26	47.85 $\pm$ 0.97	0.000	0.062	0.000	0.062
RNC	0.7273 $\pm$ 0.0104	91.10 $\pm$ 0.35	77.76 $\pm$ 0.87	5.560 $\pm$ 0.217	80.20 $\pm$ 0.68	77.73 $\pm$ 0.89	0.000	0.062	0.000	0.062
KNFST	0.3943 $\pm$ 0.0253	80.36 $\pm$ 0.85	50.90 $\pm$ 2.13	12.275 $\pm$ 0.533	49.95 $\pm$ 3.06	48.44 $\pm$ 2.99	0.000	0.062	0.000	0.062
LGBM	0.9388 $\pm$ 0.0070	98.04 $\pm$ 0.22	95.09 $\pm$ 0.56	1.228 $\pm$ 0.140	95.17 $\pm$ 0.59	95.10 $\pm$ 0.56	0.004	0.062	0.426	0.625
MLP	0.8100 $\pm$ 0.0203	93.90 $\pm$ 0.65	84.75 $\pm$ 1.63	3.812 $\pm$ 0.407	84.96 $\pm$ 1.78	84.73 $\pm$ 1.71	0.000	0.062	0.000	0.062
TabNet	0.8602 $\pm$ 0.0138	95.40 $\pm$ 0.46	88.49 $\pm$ 1.16	2.878 $\pm$ 0.290	90.10 $\pm$ 1.06	88.58 $\pm$ 1.24	0.001	0.062	0.000	0.062
SAINT	0.8864 $\pm$ 0.0050	96.29 $\pm$ 0.16	90.73 $\pm$ 0.39	2.317 $\pm$ 0.097	91.70 $\pm$ 0.48	90.81 $\pm$ 0.40	0.027	0.062	0.000	0.062
CPAI-OvA	0.9040 $\pm$ 0.0073	97.43 $\pm$ 0.20	76.90 $\pm$ 0.49	1.544 $\pm$ 0.118	77.33 $\pm$ 0.40	77.06 $\pm$ 0.44	—	—	—	—
CPAI-OvR	0.9422 $\pm$ 0.0048	98.15 $\pm$ 0.15	95.37 $\pm$ 0.38	1.158 $\pm$ 0.095	95.40 $\pm$ 0.39	95.38 $\pm$ 0.38	—	—	—	—

Table 14

Per-dataset baseline comparison on ToN-IoT.

Model	MCC $\uparrow$	ACC $\uparrow$	TPR $\uparrow$	FPR $\downarrow$	PPV $\uparrow$	F1 $\uparrow$	p_t OvA	p_w OvA	p_t OvR	p_w OvR
KNN	0.9671 $\pm$ 0.0049	99.30 $\pm$ 0.11	96.84 $\pm$ 0.66	0.402 $\pm$ 0.060	97.18 $\pm$ 0.52	96.99 $\pm$ 0.58	0.013	0.062	0.000	0.062
LDA	0.8810 $\pm$ 0.0089	97.45 $\pm$ 0.19	87.89 $\pm$ 1.07	1.460 $\pm$ 0.110	88.36 $\pm$ 1.02	87.90 $\pm$ 1.02	0.000	0.062	0.000	0.062
NuSVC	0.9014 $\pm$ 0.0073	97.89 $\pm$ 0.16	88.99 $\pm$ 1.04	1.205 $\pm$ 0.089	91.94 $\pm$ 0.76	90.16 $\pm$ 0.91	0.000	0.062	0.000	0.062
SGD	0.8632 $\pm$ 0.0206	97.02 $\pm$ 0.47	83.10 $\pm$ 4.57	1.704 $\pm$ 0.270	87.51 $\pm$ 3.37	83.35 $\pm$ 3.80	0.000	0.062	0.000	0.062
GauNB	0.4888 $\pm$ 0.0070	88.26 $\pm$ 0.15	54.26 $\pm$ 0.76	6.710 $\pm$ 0.084	56.16 $\pm$ 1.88	44.46 $\pm$ 1.44	0.000	0.062	0.000	0.062
NC	0.3579 $\pm$ 0.0199	84.97 $\pm$ 0.47	46.64 $\pm$ 1.57	8.586 $\pm$ 0.270	53.61 $\pm$ 2.44	37.11 $\pm$ 2.28	0.000	0.062	0.000	0.062
RNC	0.9138 $\pm$ 0.0074	98.14 $\pm$ 0.16	90.44 $\pm$ 0.88	1.062 $\pm$ 0.092	94.56 $\pm$ 0.47	92.11 $\pm$ 0.75	0.000	0.062	0.000	0.062
KNFST	0.9193 $\pm$ 0.0065	98.26 $\pm$ 0.14	93.28 $\pm$ 0.69	0.996 $\pm$ 0.081	91.85 $\pm$ 0.64	92.29 $\pm$ 0.71	0.000	0.062	0.000	0.062
LGBM	0.9889 $\pm$ 0.0034	99.76 $\pm$ 0.07	98.92 $\pm$ 0.35	0.135 $\pm$ 0.041	98.94 $\pm$ 0.33	98.91 $\pm$ 0.34	0.000	0.062	0.296	0.625
MLP	0.9690 $\pm$ 0.0042	99.34 $\pm$ 0.09	97.17 $\pm$ 0.34	0.379 $\pm$ 0.052	97.23 $\pm$ 0.54	97.18 $\pm$ 0.43	0.019	0.062	0.000	0.062
TabNet	0.9652 $\pm$ 0.0062	99.25 $\pm$ 0.13	96.52 $\pm$ 0.71	0.427 $\pm$ 0.076	96.73 $\pm$ 0.48	96.57 $\pm$ 0.56	0.124	0.125	0.001	0.062
SAINT	0.9813 $\pm$ 0.0024	99.60 $\pm$ 0.05	98.24 $\pm$ 0.25	0.229 $\pm$ 0.030	98.09 $\pm$ 0.26	98.15 $\pm$ 0.24	0.000	0.062	0.000	0.062
CPAI-OvA	0.9604 $\pm$ 0.0029	99.25 $\pm$ 0.05	85.59 $\pm$ 0.31	0.424 $\pm$ 0.031	85.61 $\pm$ 0.29	85.58 $\pm$ 0.29	—	—	—	—
CPAI-OvR	0.9909 $\pm$ 0.0019	99.80 $\pm$ 0.04	99.09 $\pm$ 0.16	0.112 $\pm$ 0.023	99.07 $\pm$ 0.19	99.08 $\pm$ 0.17	—	—	—	—

Table 15

Per-dataset baseline comparison on N-BaloT.

Model	MCC↑	ACC↑	TPR↑	FPR↓	PPV↑	F1↑	p_t OvA	p_w OvA	p_t OvR	p_w OvR
KNN	0.9761±0.0048	99.53±0.10	97.87±0.43	0.267±0.053	97.91±0.42	97.86±0.43	0.320	0.438	0.001	0.062
LDA	0.8751±0.0066	97.45±0.13	88.51±0.60	1.436±0.075	89.84±0.53	86.87±0.74	0.000	0.062	0.000	0.062
NuSVC	0.9683±0.0082	99.37±0.16	97.18±0.73	0.353±0.091	97.22±0.71	97.18±0.72	0.305	0.625	0.001	0.062
SGD	0.8917±0.0385	97.79±0.82	90.06±3.69	1.243±0.461	92.18±1.88	89.76±3.86	0.012	0.062	0.005	0.062
GauNB	0.6863±0.0527	93.57±1.03	71.08±4.63	3.615±0.579	70.87±6.01	67.48±5.32	0.000	0.062	0.000	0.062
NC	0.4232±0.0168	88.03±0.31	46.13±1.41	6.733±0.176	34.86±2.49	36.41±1.96	0.000	0.062	0.000	0.062
RNC	0.6805±0.0128	92.96±0.31	68.32±1.37	3.960±0.172	88.95±0.17	70.89±1.34	0.000	0.062	0.000	0.062
KNFST	0.8954±0.0150	97.92±0.27	90.62±1.23	1.172±0.154	90.98±1.73	90.49±1.07	0.000	0.062	0.000	0.062
LGBM	0.9951±0.0025	99.90±0.05	99.57±0.23	0.054±0.028	99.57±0.22	99.57±0.23	0.001	0.062	0.491	0.625
MLP	0.9862±0.0081	99.73±0.16	98.77±0.72	0.154±0.091	98.81±0.69	98.77±0.73	0.015	0.062	0.050	0.125
TabNet	0.9611±0.0194	99.21±0.40	96.44±1.81	0.444±0.226	97.14±1.29	96.35±1.88	0.351	0.438	0.023	0.062
SAINT	0.9904±0.0049	99.81±0.10	99.14±0.44	0.107±0.055	99.17±0.42	99.14±0.44	0.006	0.062	0.160	0.188
CPAI-OvA	0.9720±0.0063	99.50±0.11	87.75±0.51	0.278±0.062	87.91±0.44	87.80±0.49	—	—	—	—
CPAI-OvR	0.9941±0.0027	99.88±0.05	99.48±0.24	0.065±0.030	99.49±0.24	99.48±0.24	—	—	—	—

Table 16

Per-dataset baseline comparison on CIC-IoT2023.

Model	MCC↑	ACC↑	TPR↑	FPR↓	PPV↑	F1↑	p_t OvA	p_w OvA	p_t OvR	p_w OvR
KNN	0.9565±0.0036	98.98±0.08	95.08±0.46	0.610±0.050	95.45±0.63	95.20±0.53	0.000	0.062	0.000	0.062
LDA	0.3268±0.0617	78.67±7.30	33.33±0.00	12.800±4.382	21.48±4.06	23.43±4.69	0.000	0.062	0.000	0.062
NuSVC	0.9100±0.0092	97.88±0.22	89.51±1.11	1.270±0.130	97.24±0.29	92.66±0.75	0.000	0.062	0.000	0.062
SGD	0.9838±0.0024	99.62±0.06	98.15±0.30	0.228±0.034	98.33±0.28	98.21±0.29	0.001	0.062	0.000	0.062
GauNB	0.9976±0.0013	99.94±0.03	99.74±0.13	0.034±0.018	99.83±0.16	99.78±0.13	0.129	0.188	0.038	0.062
NC	0.5836±0.0109	88.01±0.34	70.13±1.22	7.194±0.201	66.99±0.83	64.35±1.12	0.000	0.062	0.000	0.062
RNC	0.6867±0.0046	92.61±0.11	63.05±0.54	4.434±0.064	94.88±0.05	69.95±0.67	0.000	0.062	0.000	0.062
KNFST	0.9756±0.0100	99.43±0.24	97.61±0.71	0.344±0.142	97.74±1.04	97.63±0.93	0.010	0.062	0.006	0.062
LGBM	0.9996±0.0010	99.99±0.02	99.95±0.11	0.006±0.013	99.99±0.02	99.97±0.07	0.000	0.062	0.999	0.655
MLP	0.9981±0.0014	99.96±0.03	99.82±0.15	0.026±0.019	99.85±0.13	99.84±0.14	0.002	0.062	0.022	0.062
TabNet	0.9977±0.0028	99.95±0.06	99.80±0.30	0.032±0.039	99.75±0.29	99.77±0.30	0.323	0.438	0.246	0.125
SAINT	0.9996±0.0010	99.99±0.02	99.95±0.11	0.006±0.013	99.95±0.11	99.95±0.11	0.000	0.062	0.999	0.655
CPAI-OvA	0.9961±0.0012	99.91±0.03	99.60±0.17	0.054±0.017	99.55±0.14	99.58±0.15	—	—	—	—
CPAI-OvR	0.9996±0.0006	99.99±0.01	99.95±0.07	0.006±0.009	99.95±0.07	99.95±0.07	—	—	—	—

Table 17

Per-dataset baseline comparison on Edge-IIoTset.

Model	MCC↑	ACC↑	TPR↑	FPR↓	PPV↑	F1↑	p_t OvA	p_w OvA	p_t OvR	p_w OvR
KNN	0.7234±0.0107	96.52±0.14	73.93±1.02	1.862±0.073	77.24±0.65	73.58±0.96	0.000	0.062	0.000	0.062
LDA	0.5426±0.0053	94.23±0.06	56.71±0.47	3.092±0.033	56.18±0.94	53.60±0.46	0.000	0.062	0.000	0.062
NuSVC	0.6819±0.0062	96.01±0.08	70.06±0.59	2.139±0.042	71.04±0.62	68.59±0.65	0.000	0.062	0.000	0.062
SGD	0.5326±0.0205	93.90±0.40	54.27±3.00	3.267±0.214	58.32±3.38	50.37±3.94	0.001	0.062	0.000	0.062
GauNB	0.4385±0.0123	92.69±0.15	45.17±1.16	3.916±0.083	41.79±2.03	38.26±1.28	0.000	0.062	0.000	0.062
NC	0.4096±0.0121	92.51±0.15	43.84±1.11	4.011±0.079	40.61±0.74	38.77±0.97	0.000	0.062	0.000	0.062
RNC	0.6202±0.0078	95.24±0.10	64.30±0.73	2.550±0.052	65.06±0.94	62.99±0.80	0.372	0.625	0.000	0.062
KNFST	0.3641±0.0261	91.82±0.27	38.68±2.05	4.380±0.146	36.53±2.47	32.55±1.37	0.000	0.062	0.000	0.062
LGBM	0.8889±0.0033	98.61±0.04	89.61±0.31	0.742±0.022	89.98±0.29	89.63±0.30	0.000	0.062	0.013	0.062
MLP	0.7703±0.0163	97.12±0.21	78.42±1.56	1.541±0.111	79.33±1.40	77.88±1.70	0.000	0.062	0.000	0.062
TabNet	0.6961±0.0851	96.19±1.08	71.39±8.13	2.043±0.580	74.79±5.35	70.53±8.85	0.136	0.188	0.008	0.062
SAINT	0.8441±0.0097	98.05±0.12	85.38±0.92	1.044±0.066	86.28±0.82	85.35±0.95	0.000	0.062	0.001	0.062
CPAI-OvA	0.6261±0.0060	95.61±0.07	60.81±0.52	2.342±0.037	61.29±0.52	59.15±0.57	—	—	—	—
CPAI-OvR	0.8838±0.0035	98.55±0.04	89.13±0.33	0.776±0.023	89.48±0.25	89.15±0.32	—	—	—	—

Table 18

Per-dataset baseline comparison on 5G-NIDD.

Model	MCC↑	ACC↑	TPR↑	FPR↓	PPV↑	F1↑	p_t OvA	p_w OvA	p_t OvR	p_w OvR
KNN	0.9854±0.0014	99.71±0.03	98.70±0.12	0.163±0.015	98.72±0.13	98.70±0.12	0.000	0.062	0.000	0.062
LDA	0.9060±0.0048	98.08±0.10	91.34±0.45	1.082±0.056	93.20±0.38	90.88±0.49	0.000	0.062	0.000	0.062
NuSVC	0.9421±0.0060	98.84±0.12	94.79±0.55	0.651±0.068	95.34±0.42	94.80±0.54	0.002	0.062	0.000	0.062
SGD	0.9671±0.0102	99.34±0.21	97.04±0.94	0.369±0.117	97.30±0.75	97.02±0.96	0.711	0.812	0.004	0.062
GauNB	0.8574±0.0043	97.12±0.08	87.02±0.37	1.622±0.047	88.69±0.56	86.37±0.39	0.000	0.062	0.000	0.062
NC	0.7491±0.0022	94.80±0.05	76.61±0.22	2.924±0.027	83.02±0.15	72.70±0.16	0.000	0.062	0.000	0.062
RNC	0.8761±0.0077	97.40±0.16	88.30±0.74	1.463±0.092	93.46±0.41	88.71±0.68	0.000	0.062	0.000	0.062
KNFST	0.9134±0.0108	98.29±0.22	92.29±0.97	0.964±0.121	92.47±0.91	92.32±0.96	0.000	0.062	0.000	0.062
LGBM	0.9921±0.0020	99.84±0.04	99.30±0.18	0.087±0.022	99.31±0.18	99.30±0.18	0.000	0.062	0.074	0.125
MLP	0.9876±0.0022	99.76±0.04	98.90±0.19	0.137±0.024	98.92±0.20	98.90±0.19	0.000	0.062	0.002	0.062
TabNet	0.9812±0.0063	99.63±0.13	98.32±0.57	0.210±0.071	98.42±0.48	98.33±0.56	0.010	0.062	0.009	0.062
SAINT	0.9884±0.0027	99.77±0.05	98.97±0.24	0.129±0.031	98.99±0.23	98.97±0.24	0.001	0.062	0.009	0.062
CPAI-OvA	0.9690±0.0031	99.40±0.08	95.28±4.22	0.339±0.045	95.37±4.21	95.26±4.20	—	—	—	—
CPAI-OvR	0.9950±0.0009	99.90±0.02	99.56±0.08	0.056±0.010	99.56±0.08	99.56±0.08	—	—	—	—